Index

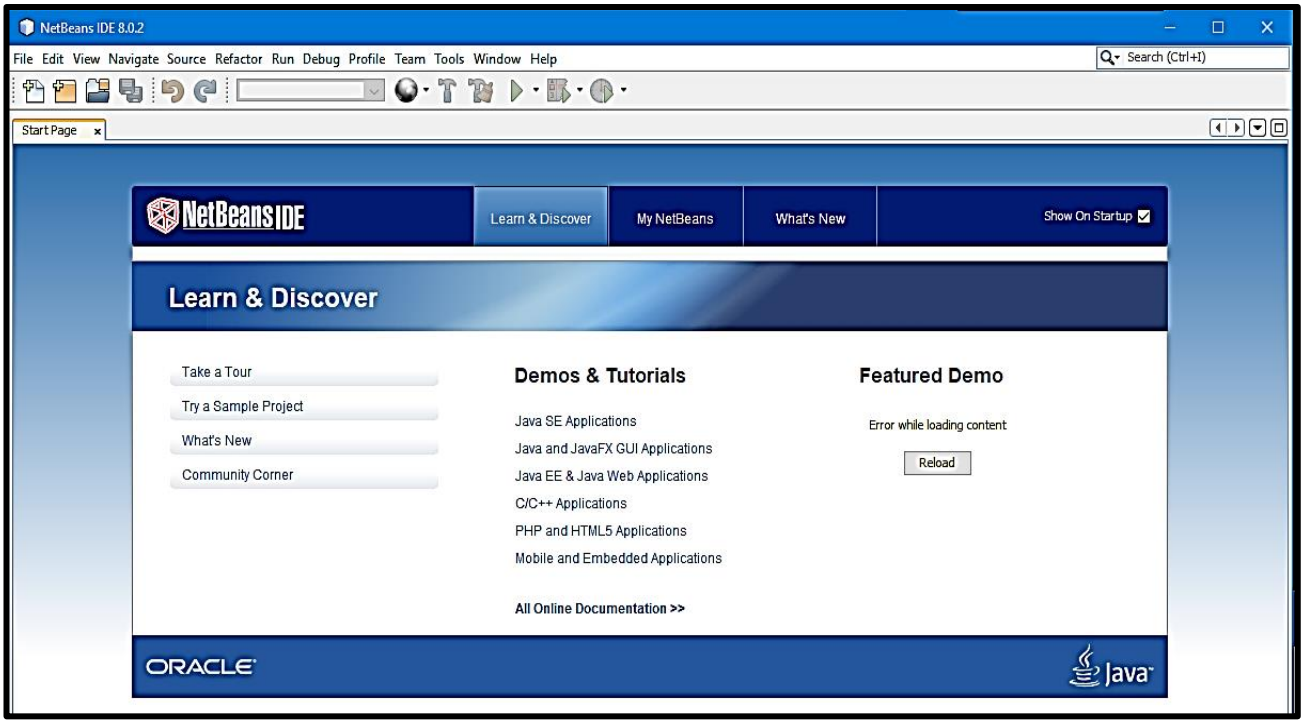
Sr. No.	Name of the Practicals	Date	Sign.
1	Define a simple service like Converting Rs into Dollar and Call it from different platform like JAVA and .NET.		
2	Create a Simple SOAP service.		
3	Create a Simple REST Service.		
4	Develop application to consume Google's search / Google's Map RESTful Web service.		
5	Installation and Configuration of virtualization using KVM.		
6	Develop application to download image/video from server or upload image/video to server using MTOM techniques.		
7	Using AWS Flow Framework develop application that includes a simple workflow. Workflow calls an activity to print hello world to the console. It must define the basic usage of AWS Flow Framework, including defining contracts, implementation of activities and workflow coordination logic and worker programs to host them.		

Practical 1

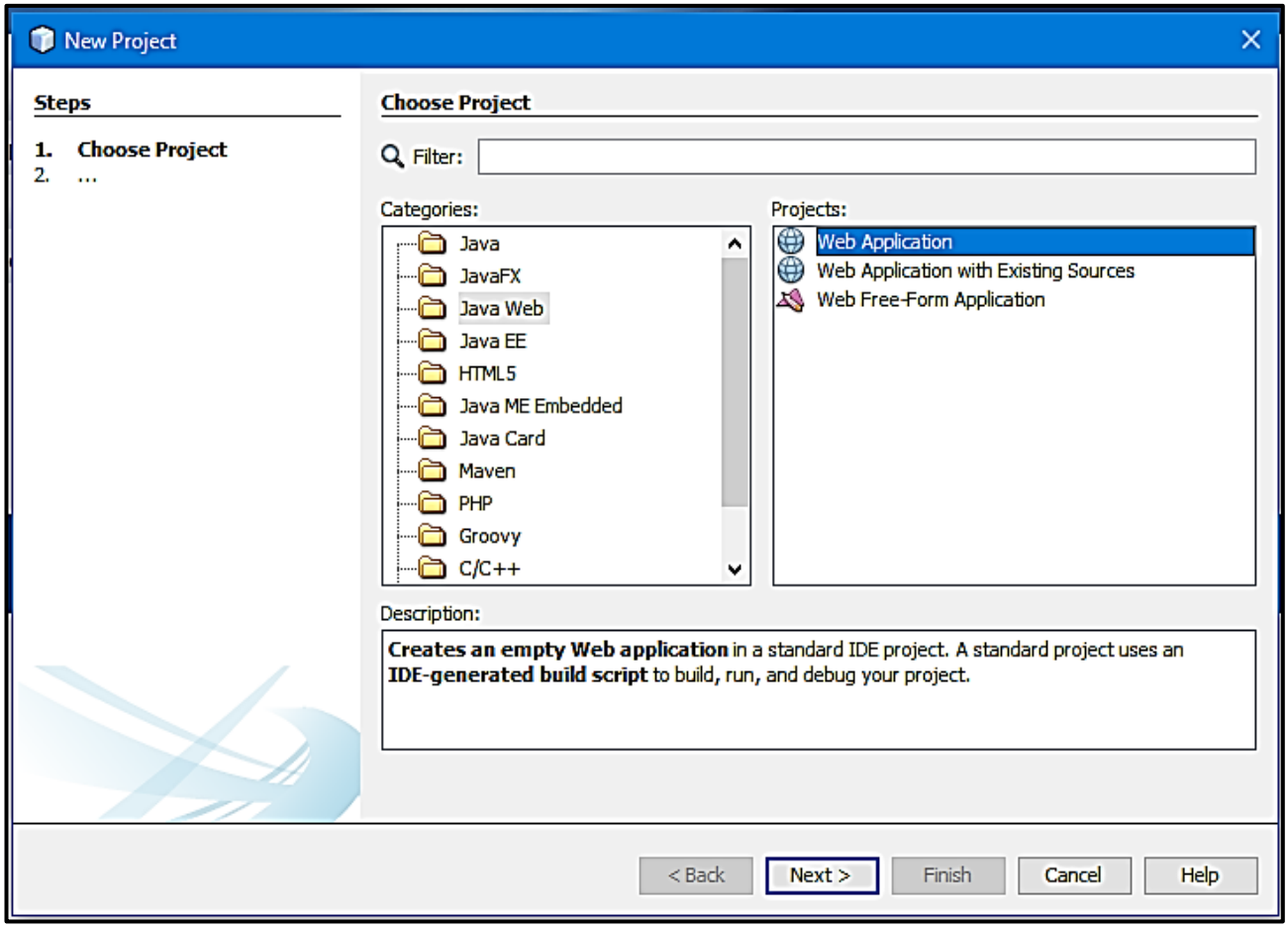
<u>Aim:</u> Define a simple service like Converting Rs into Dollar and Call it from different platform like JAVA and .NET.

<u>Steps:</u>

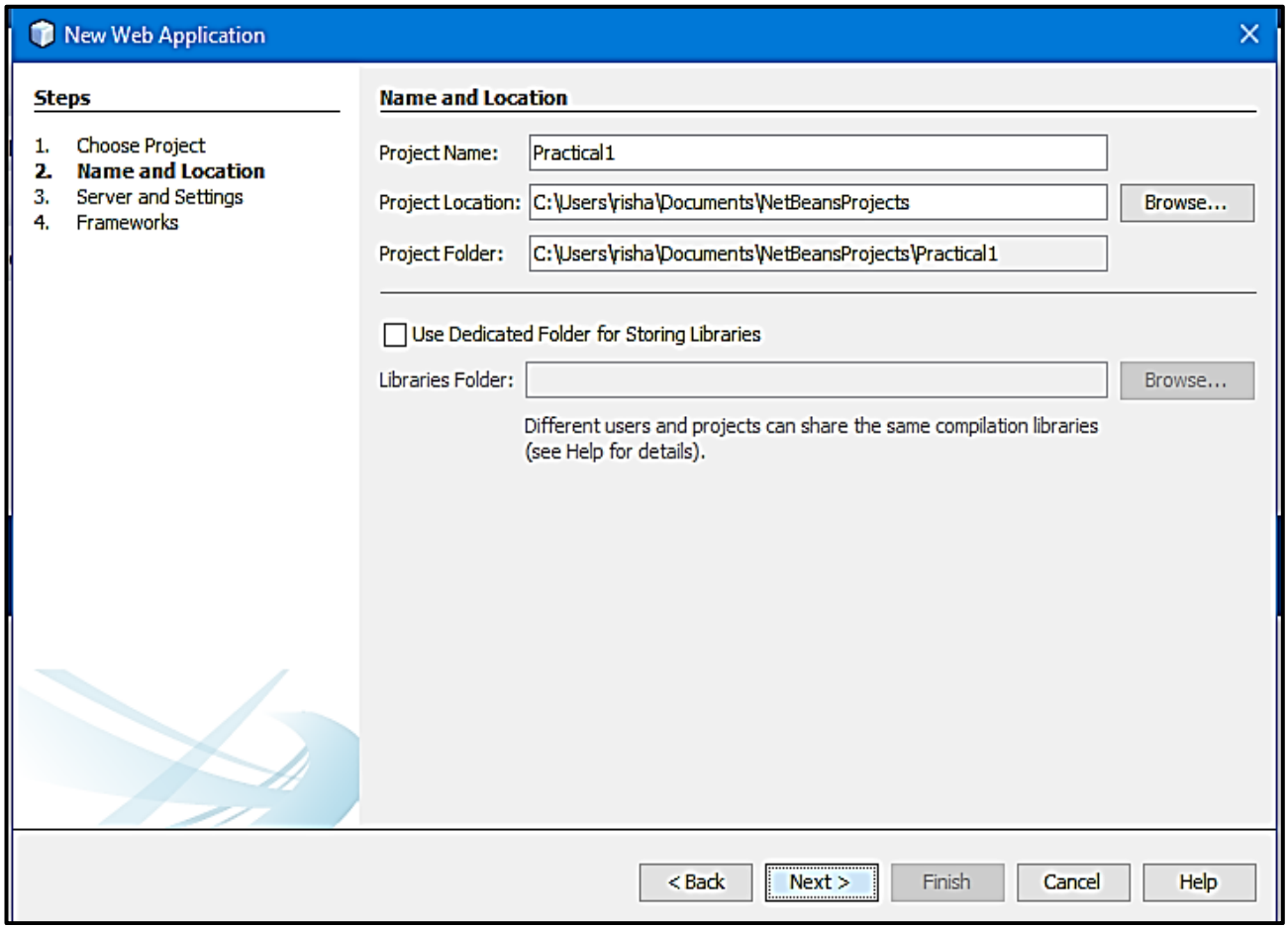
1] Open the NetBeans, and you will get the following screen. Close the start page.



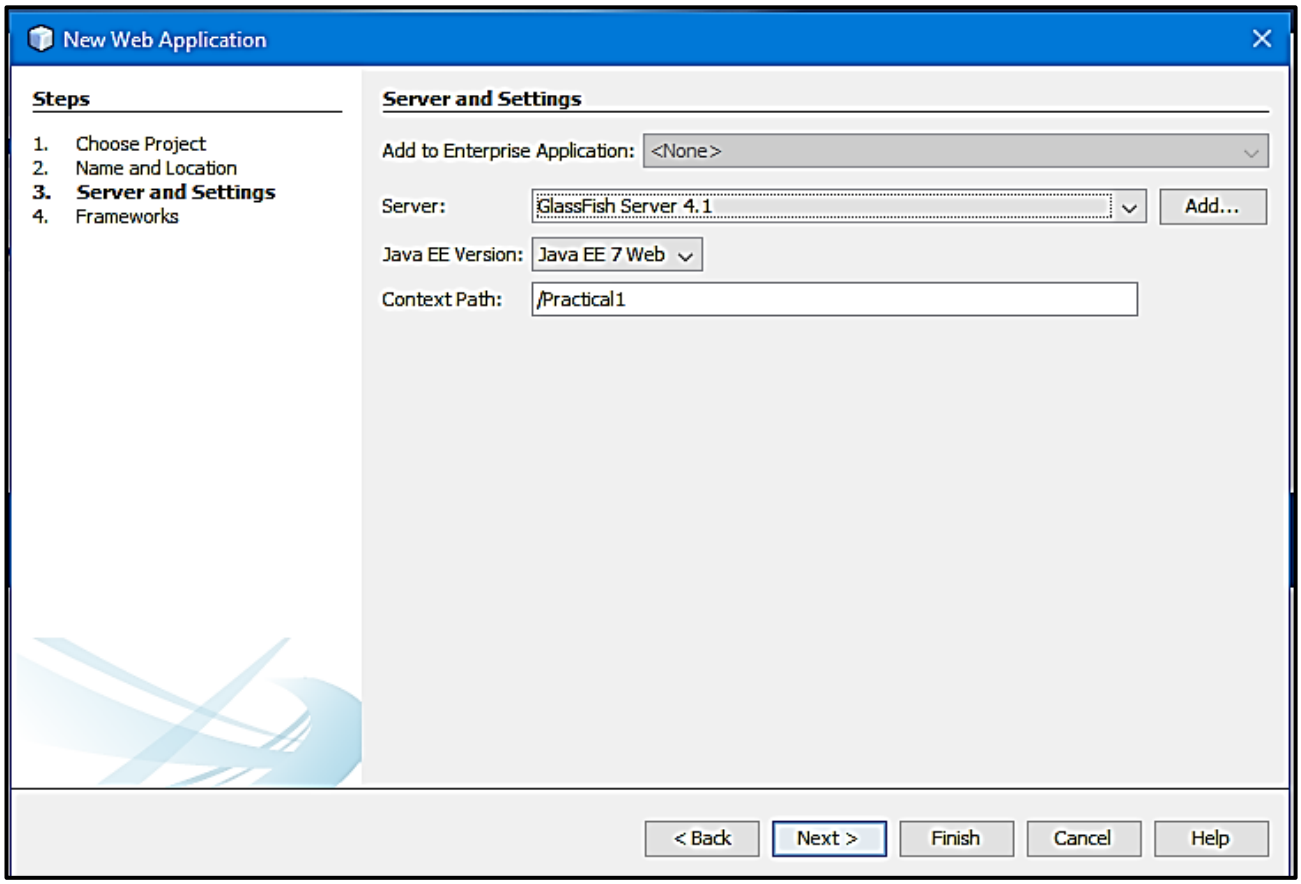
2] Now click on the file tab and click on new project you will get the following screen:



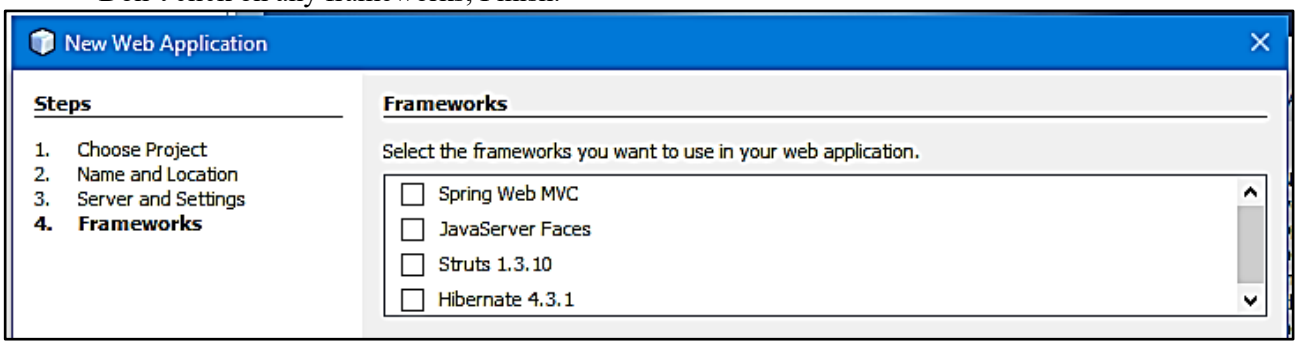
3] In Categories select Java Web and in Projects, Select Web Application. After selecting click on next. You will get the following window:



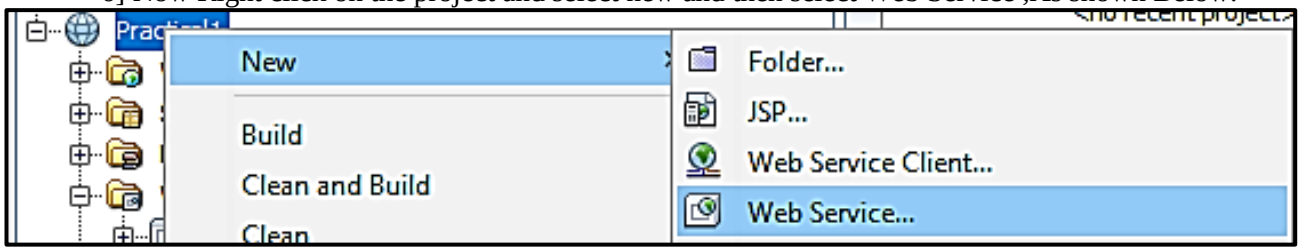
4] Now Give name to the Project Name, and click on next. You will get the following window then again, click on finish:



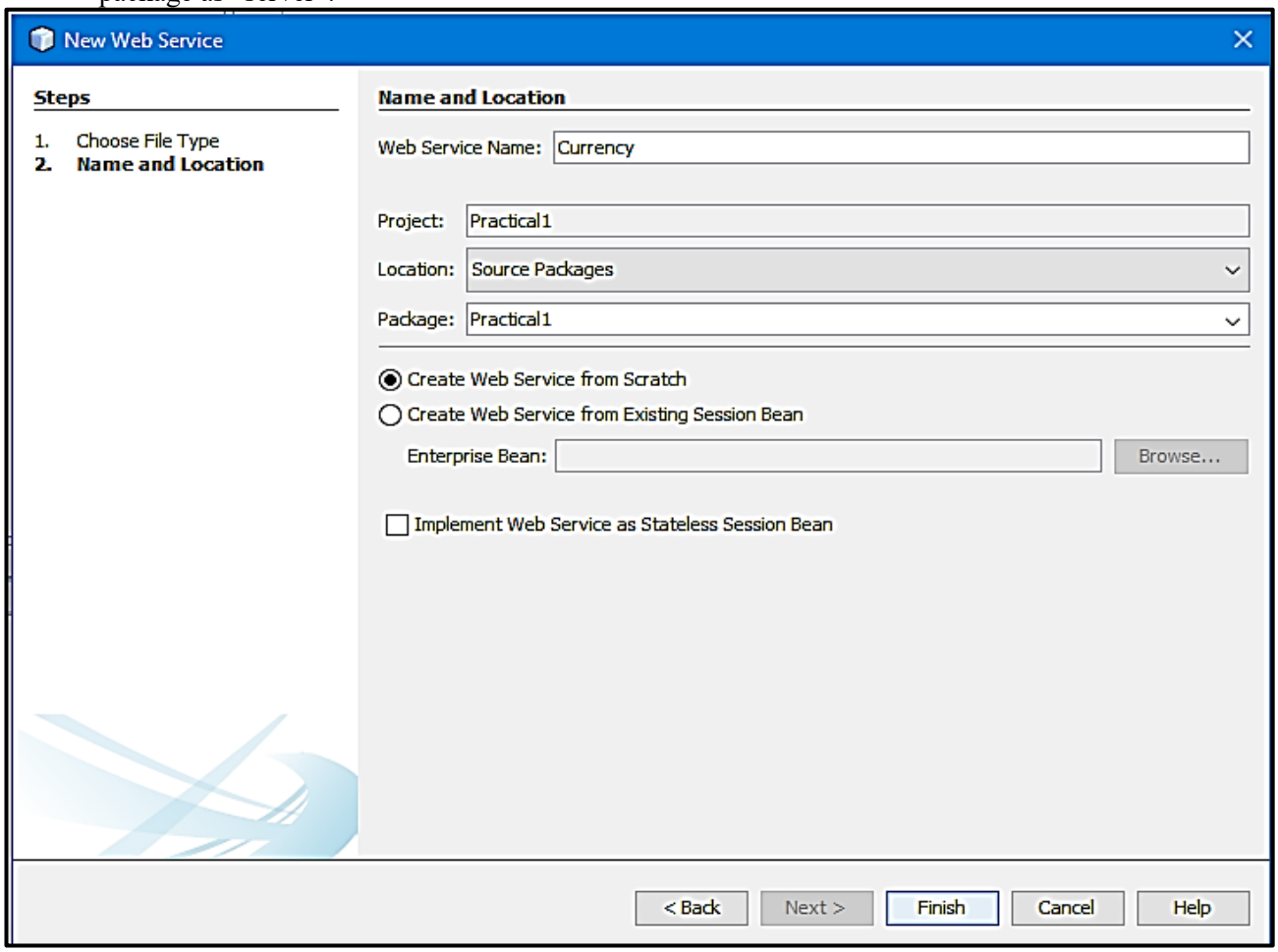
Don’t click on any frameworks, Finish.



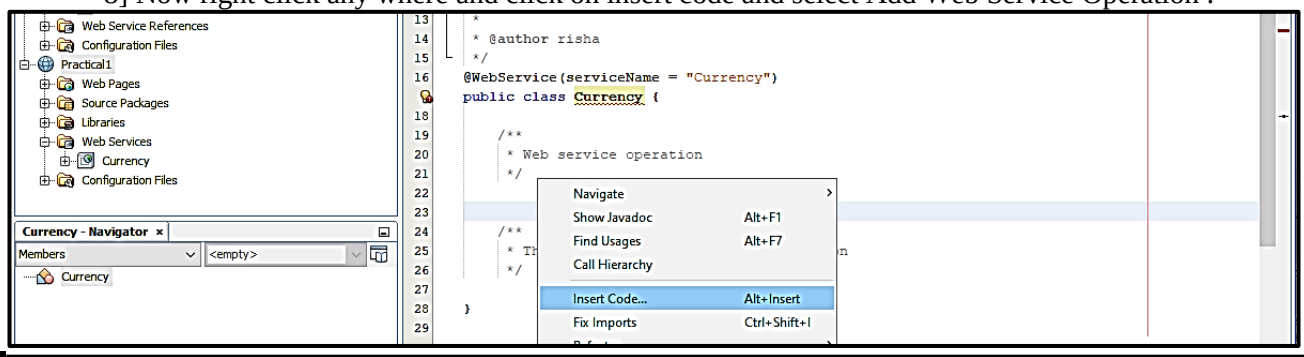
6] Now Right click on the project and select new and then select Web Service ,As shown Below:



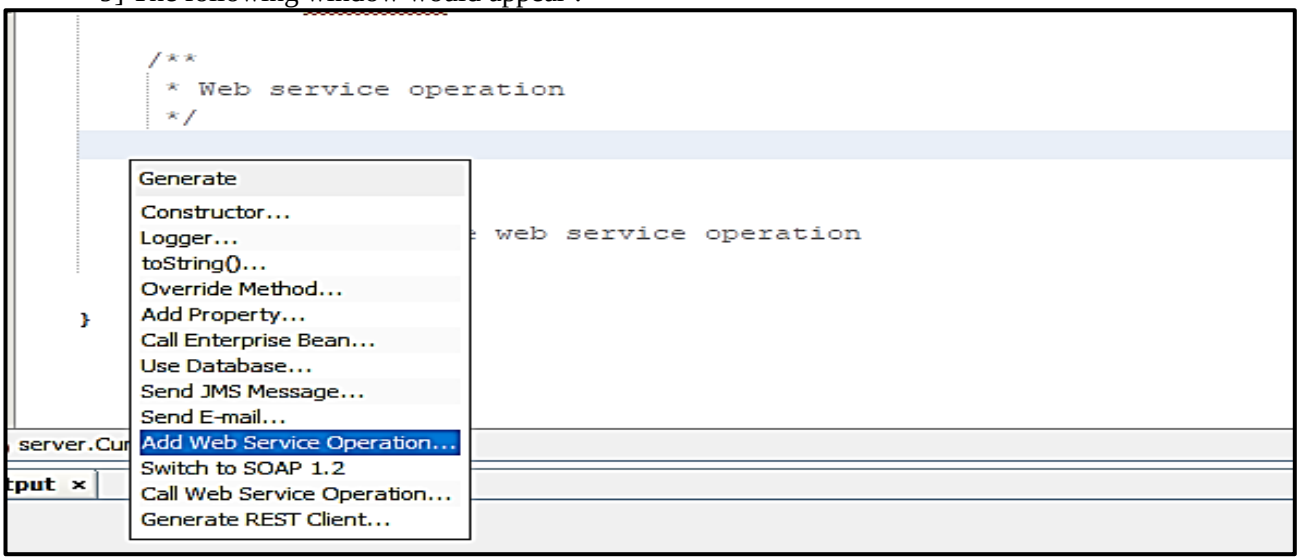
7] After clicking Web Service following window should appear, now give name to web service and give package as “server”.



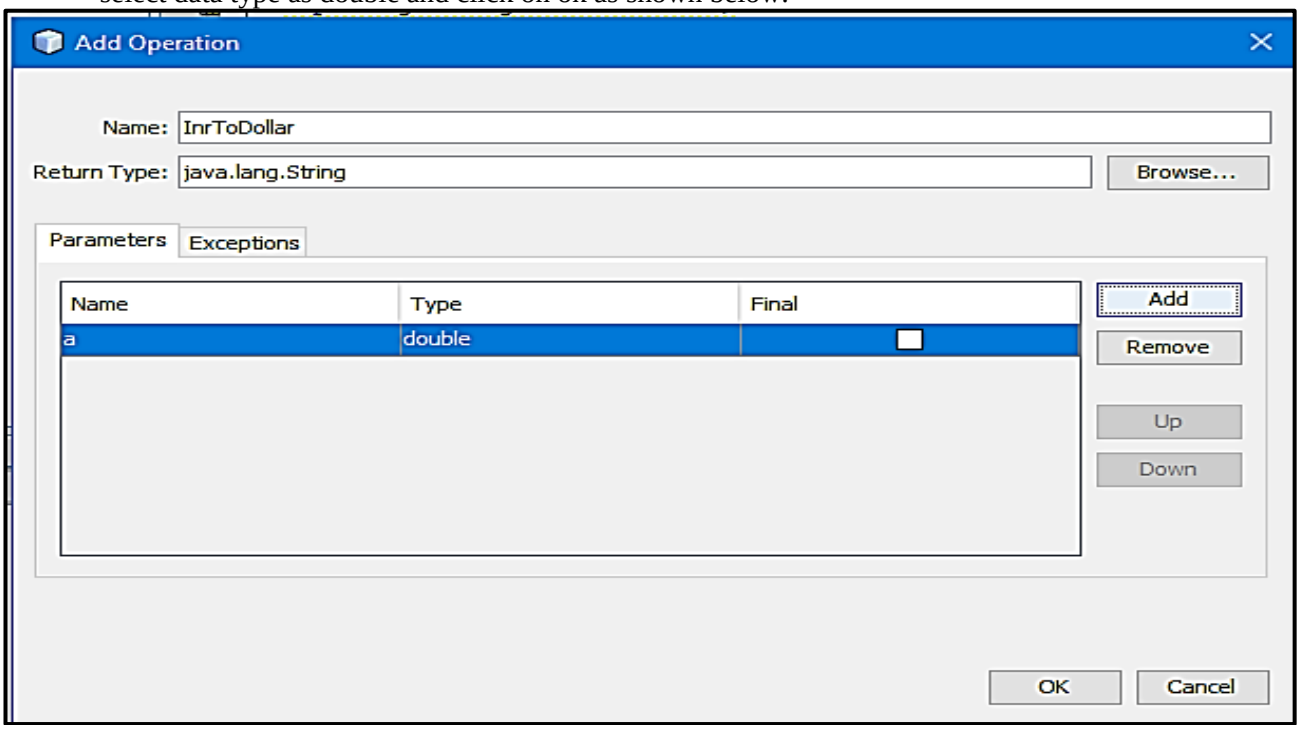
8] Now right click any where and click on insert code and select Add Web Service Operation :



9] The following window would appear :



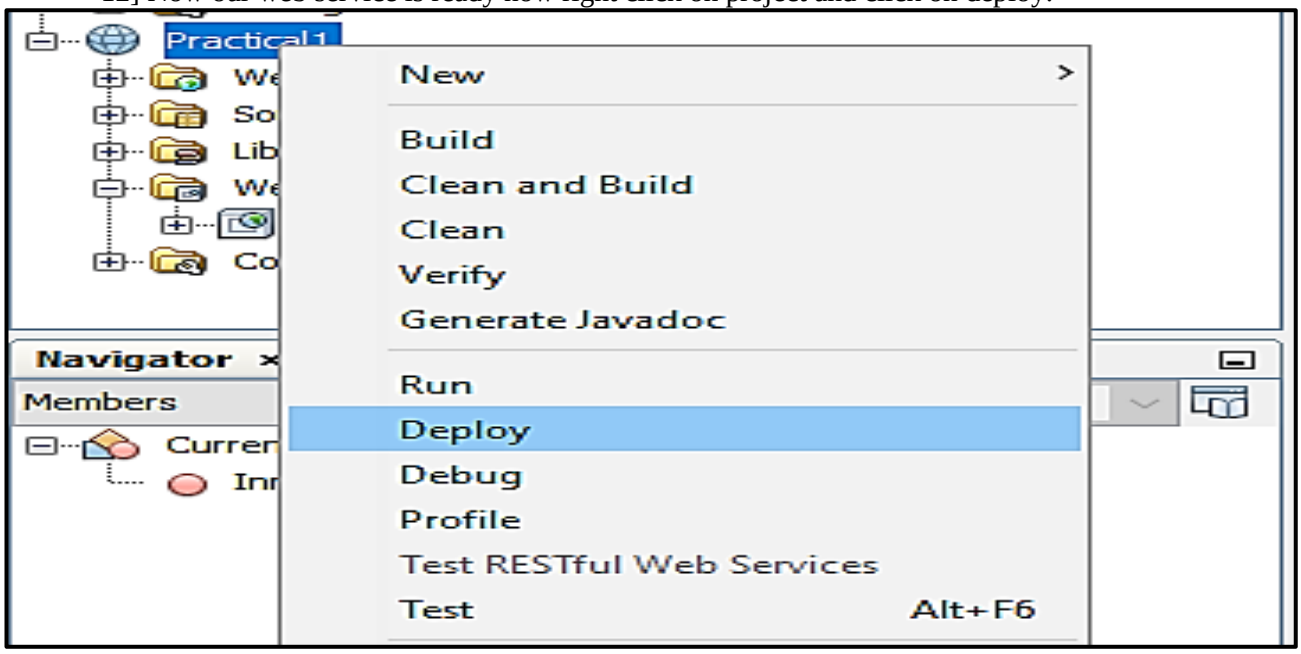
10] Just give name to the method or operation and click on add button to add parameters to the method as here we are converting dollar to rupees we should need only one parameter. give the name to the variable select data type as double and click on ok as shown below:



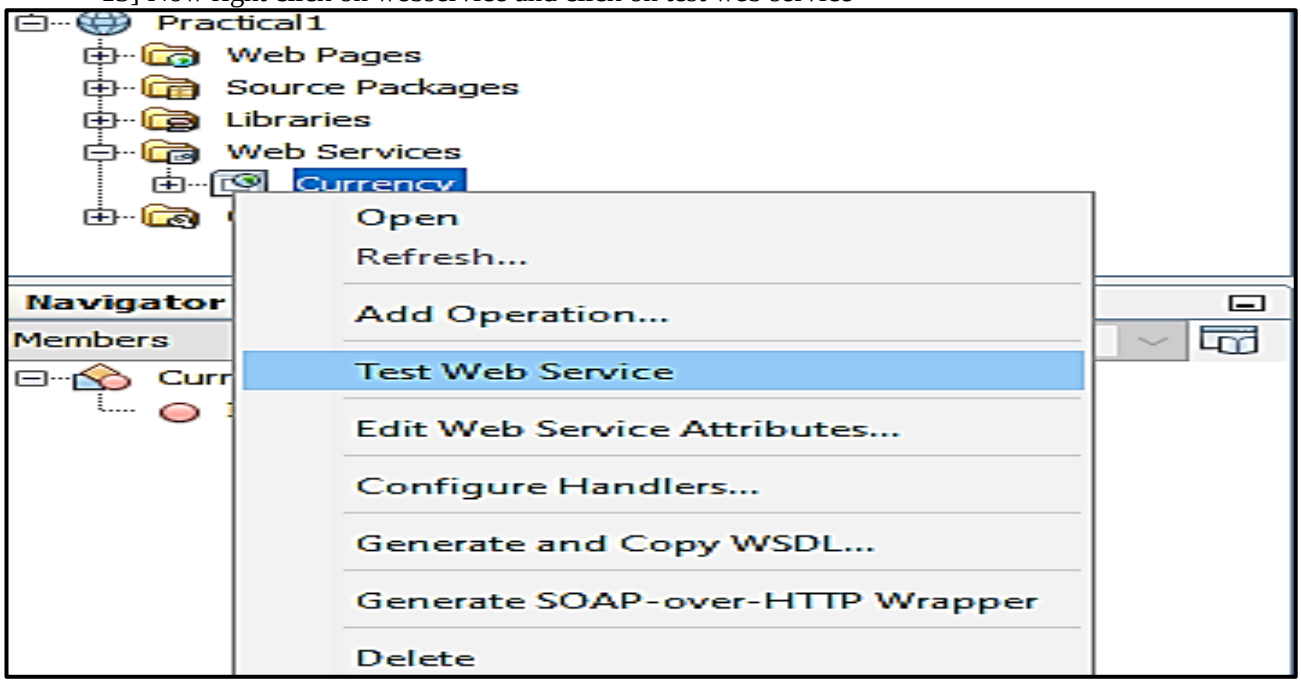
11] After clicking ok code will autogenerate, make changes in that code as mentioned below:

```
package server;

import javax.jws.WebService;
import javax.jws.WebMethod;
import javax.jws.WebParam;

@WebService(serviceName = "Currency")
public class Currency {

    @WebMethod(operationName = "InrToDollar")
    public String InrToDollar(@WebParam(name = "a") double a) {
        return "The Indian Rupees "+a+" in Dollars is"+(a/91.56);
    }

}
```

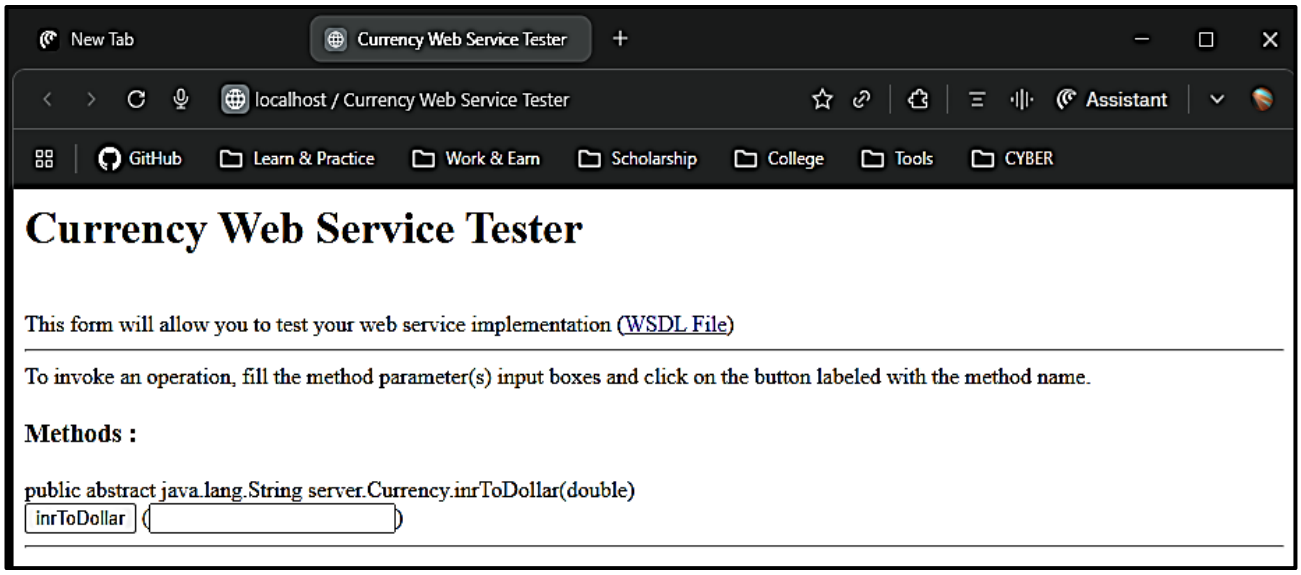
12] Now our web service is ready now right click on project and click on deploy:



13] Now right click on webservice and click on test web service



Output:



New Tab | Method invocation trace

localhost / Method invocation trace

GitHub | Learn & Practice | Work & Earn | Scholarship | College | Tools | CYBER

inrtoDollar Method invocation

Method parameter(s)

Type	Value
double	1233

Method returned

java.lang.String : **"The Indian Rupees 1233.0 in Dollar is 13.643908376673675"**

SOAP Request


```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Header/>
    <S:Body>
        <ns2:InrtoDollar xmlns:ns2="http://server/">
            <a>1233.0</a>
        </ns2:InrtoDollar>
    </S:Body>
</S:Envelope>
```

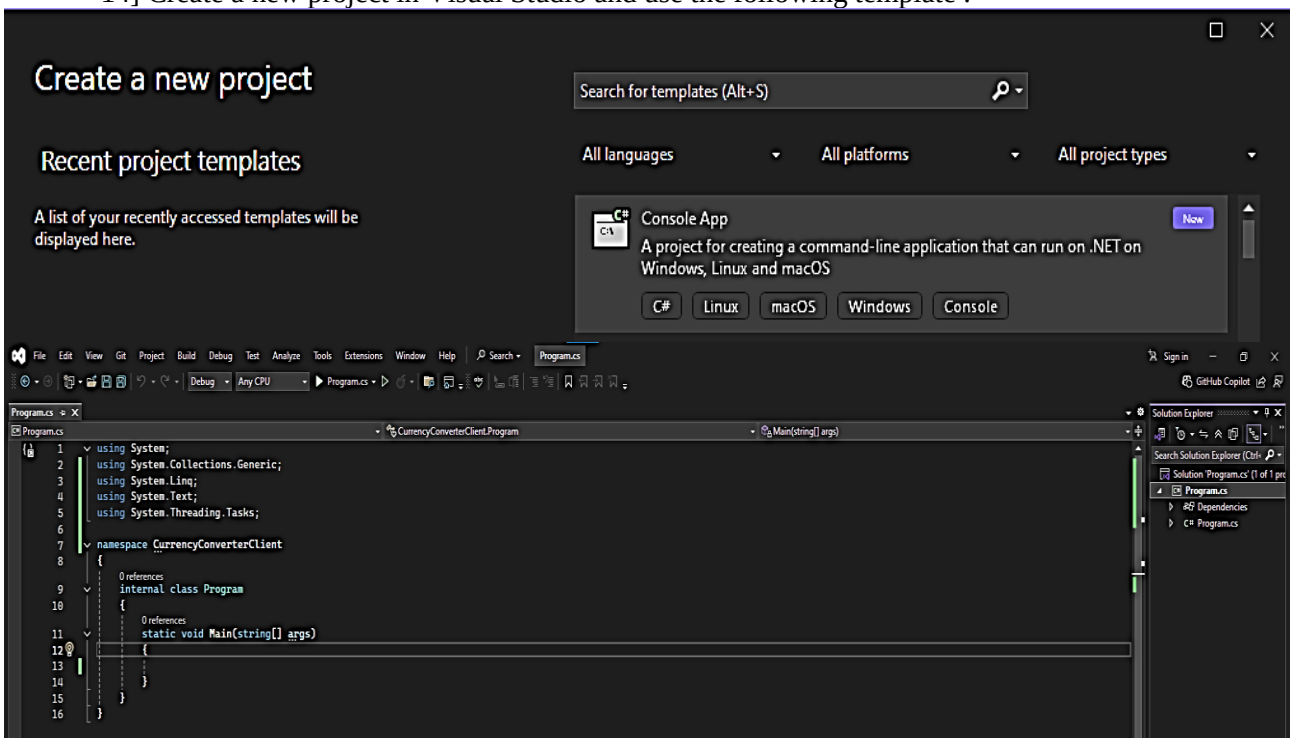

SOAP Response


```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Header/>
    <S:Body>
        <ns2:InrtoDollarResponse xmlns:ns2="http://server/">
            <return>The Indian Rupees 1233.0 in Dollar is 13.643908376673675</return>
        </ns2:InrtoDollarResponse>
    </S:Body>
</S:Envelope>
```


So this is how we created our web service and deployed it

Now Calling the Service through .NET

14] Create a new project in Visual Studio and use the following template :



15] Click on Project > Add Service Reference

Note : the service from earlier must be hosted for Service Reference to be detected

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace CurrencyConverterClient
{
    0 references
    internal class Program
    {
        0 references
        static void Main(string[] args)
        {

        }
    }
}
```

Add service reference

Select a service reference to add to your application

OpenAPI
Consume web services which conform to the OpenAPI Specification

gRPC
Consume and produce web services which conform to the gRPC open source universal RPC framework

WCF Web Service
Add a WCF web service reference to your project.

Back | Next | Finish | Cancel

Add new WCF Web Service service reference

Specify the service to add

To see a list of available services on a specific server, enter a service URL and select Go. To find available services in the solution, select Discover. To load a service metadata from a WSDL file, select Browse.

URI:

http://localhost:8080/CurrencyConversion/Currency?wsdl

Stop | Discover | Browse...

Services:

- Currency
  - Currency

Operations:

Status:

Number of services found: 1

Namespace:

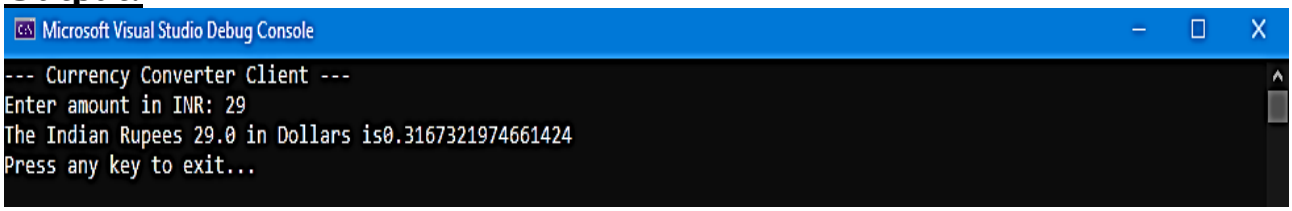
ServiceReference1

Back | Next | Finish | Cancel

16] Modify the code:

```
using ServiceReference1;
using System;
using System.Threading.Tasks;

namespace Currency_Converter
{
   internal class Program
   {

      static async Task Main(string[] args)
      {
CurrencyClient client = new CurrencyClient();

Console.WriteLine("--- Currency Converter Client ---");
Console.Write("Enter amount in INR: ");

         string input = Console.ReadLine();

         if (double.TryParse(input, out double inr))
         {
            try
            {

               var response = await client.InrToDollarAsync(inr);


Console.WriteLine(response.Body.@return);
            }
            catch (Exception ex)
            {
Console.WriteLine("Error: " + ex.Message);
            }
         }
Console.WriteLine("Press any key to exit...");
Console.ReadKey();
      }
   }
}
```

Output:

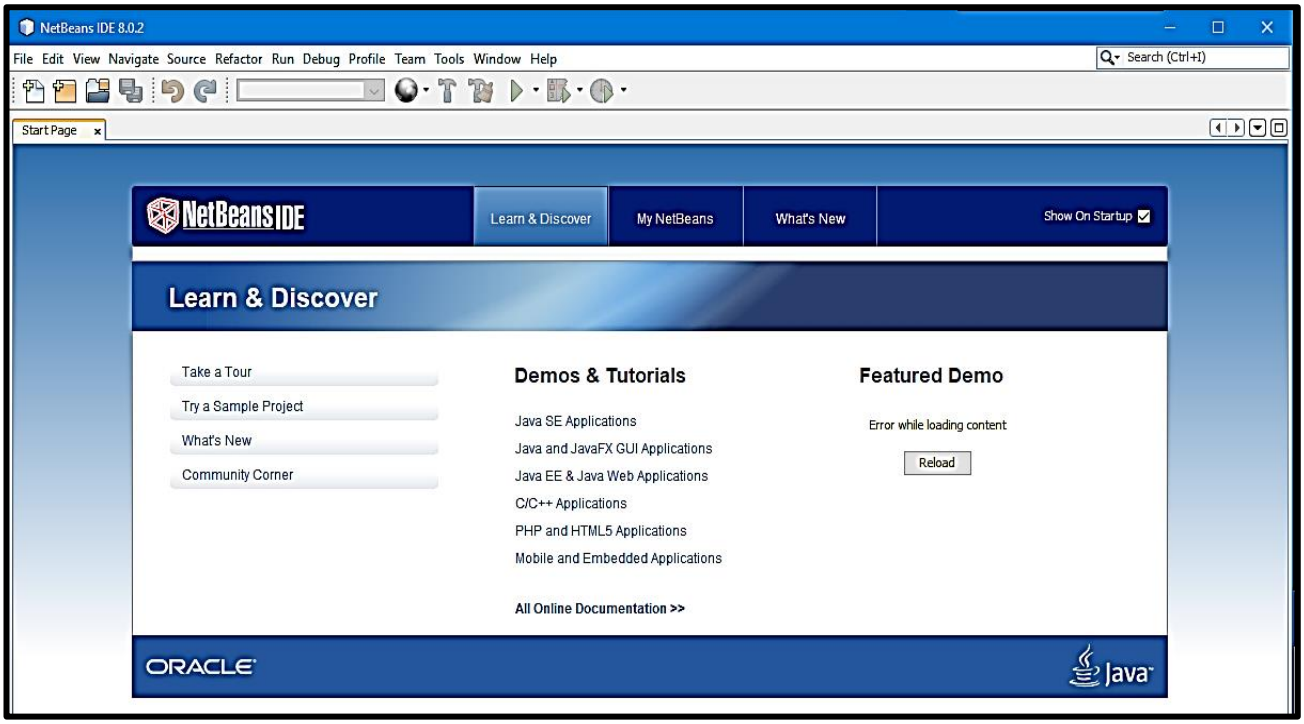


Practical 2

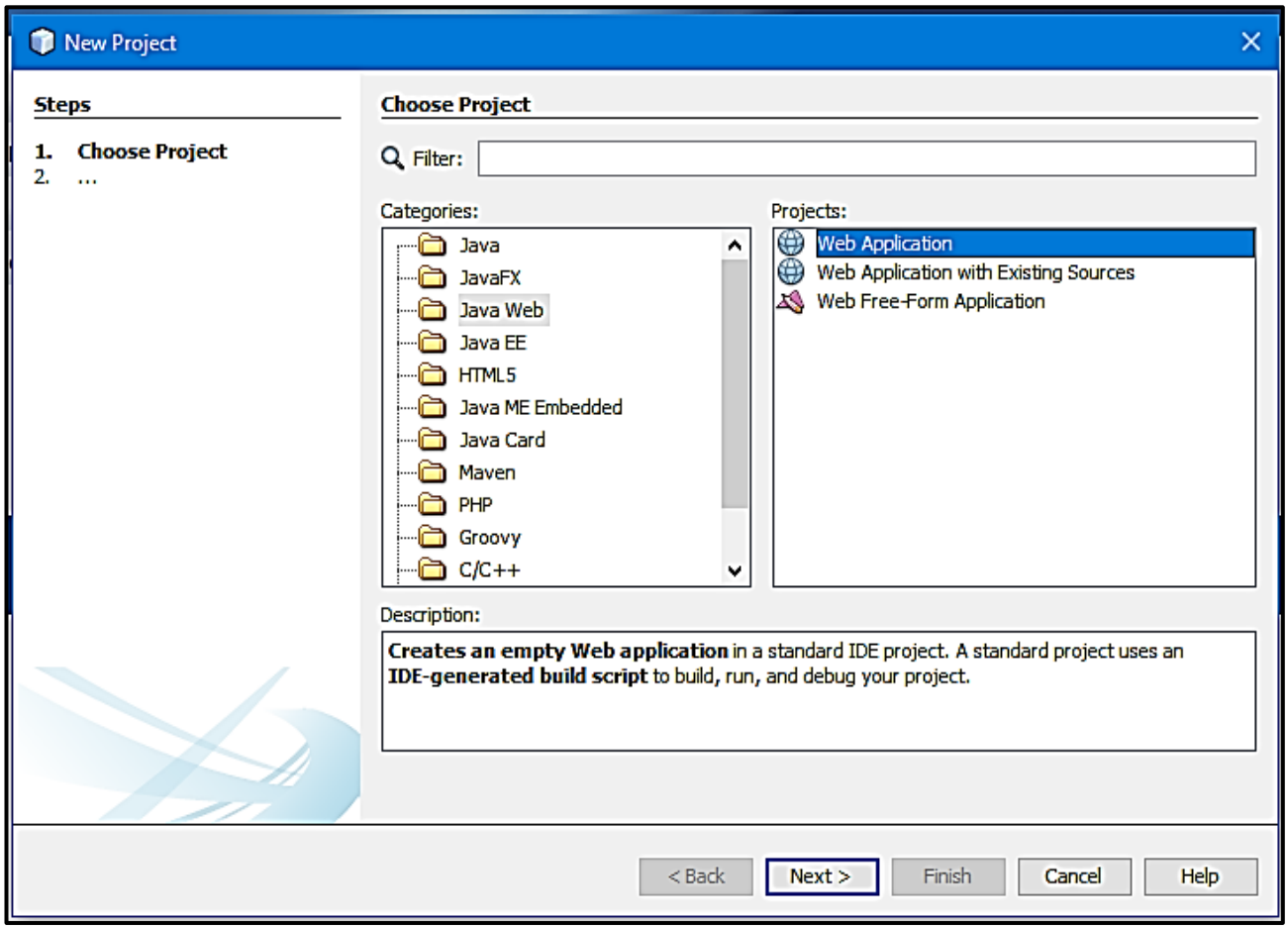
Aim: Create a Simple SOAP service.

Steps:

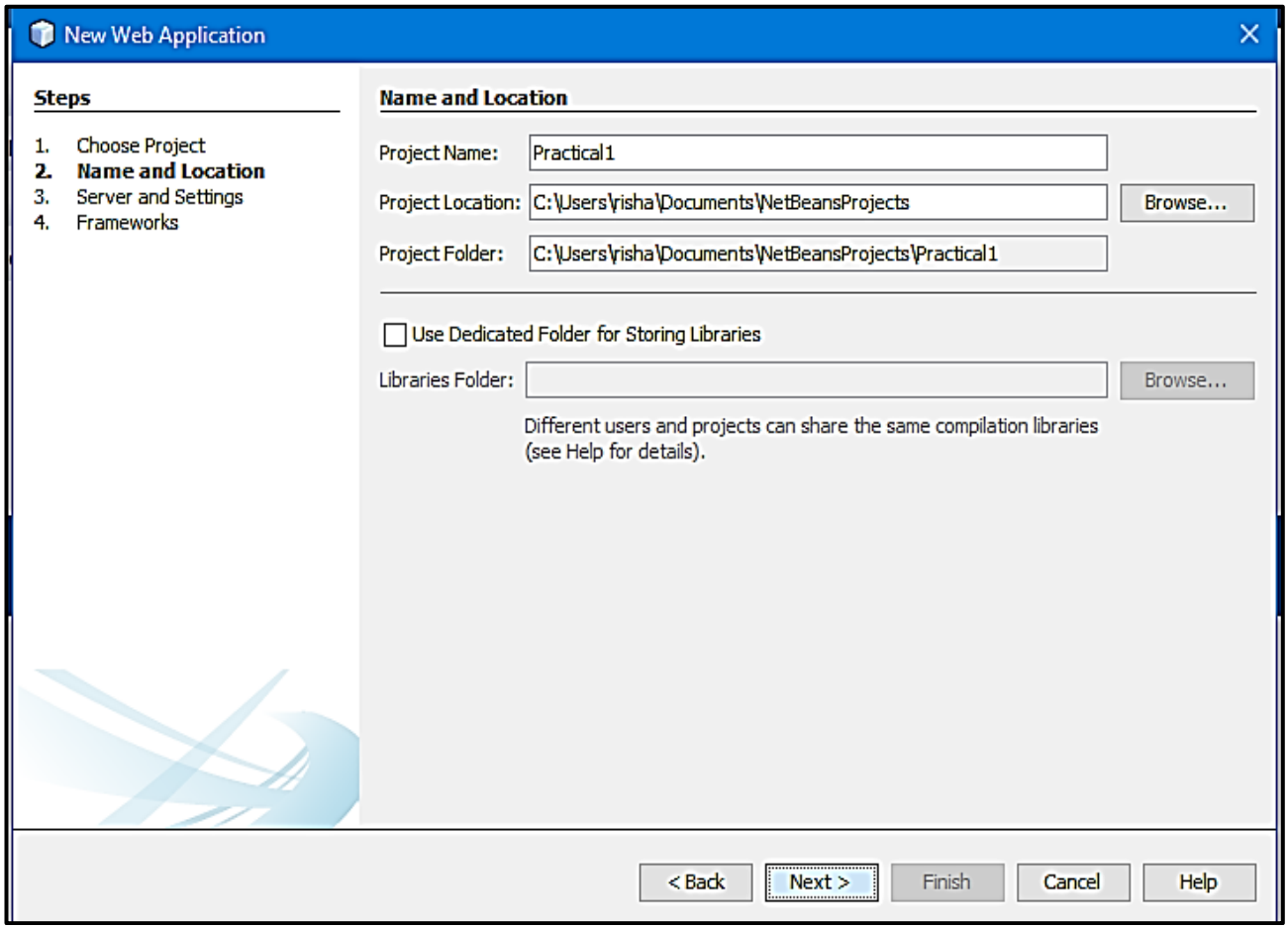
1] Open the NetBeans, and you will get the following screen. Close the start page.



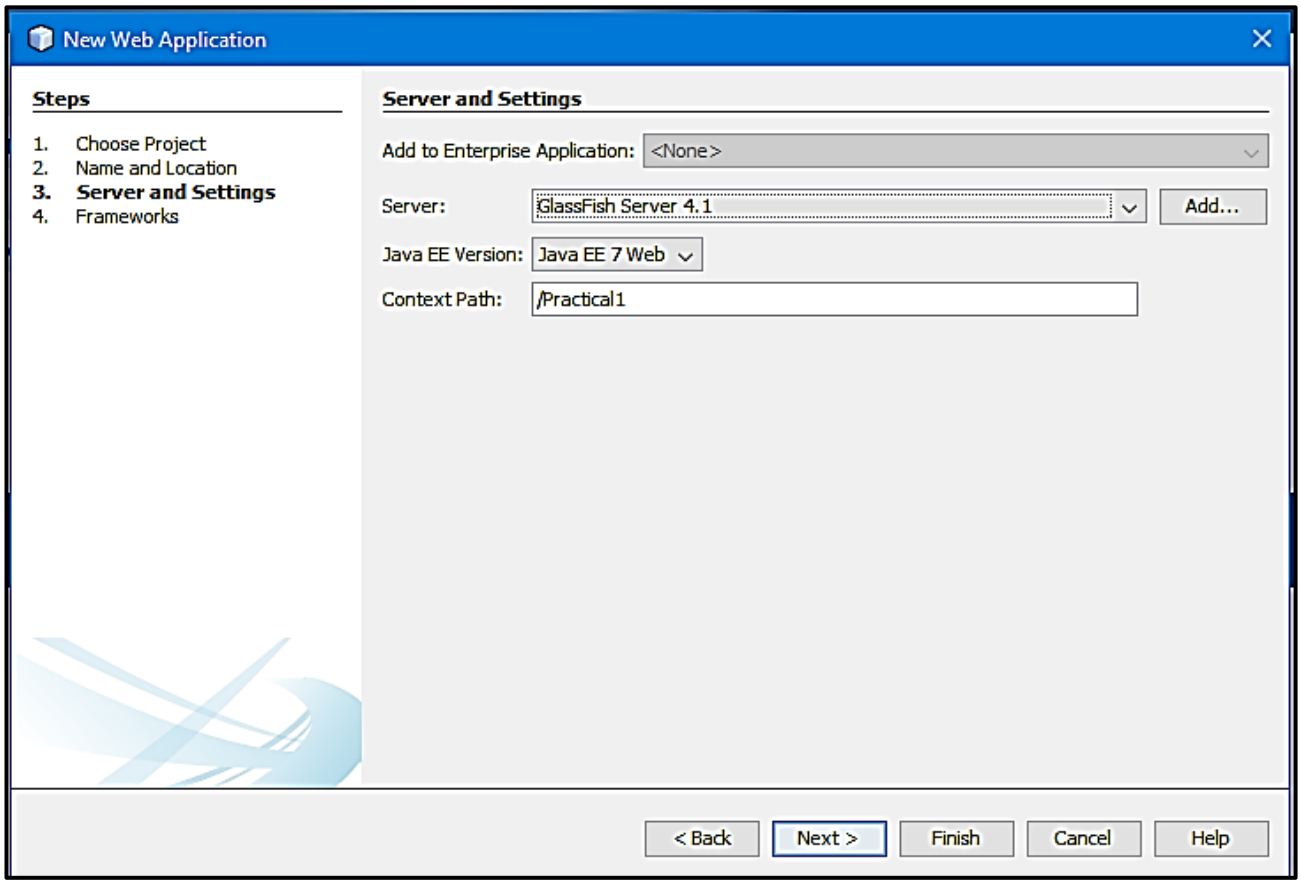
2] Now click on the file tab and click on new project you will get the following screen:



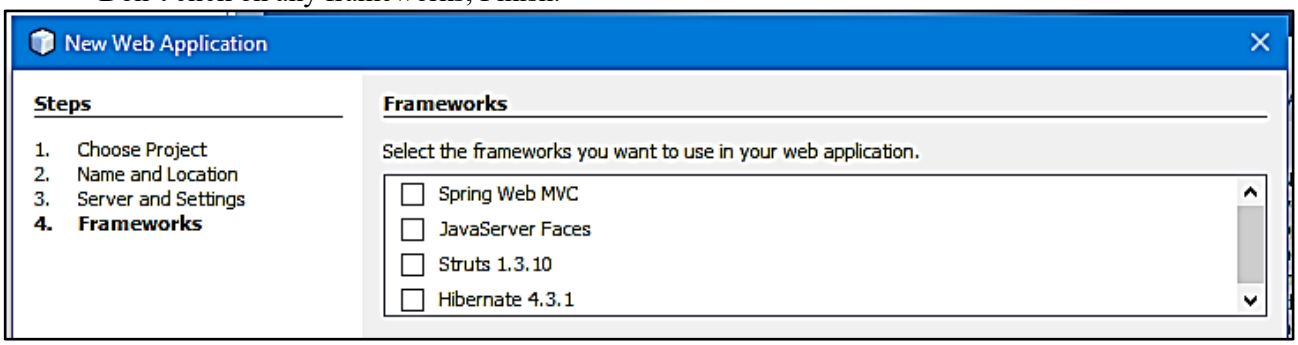
3] In Categories select Java Web and in Projects, Select Web Application. After selecting click on next. You will get the following window:



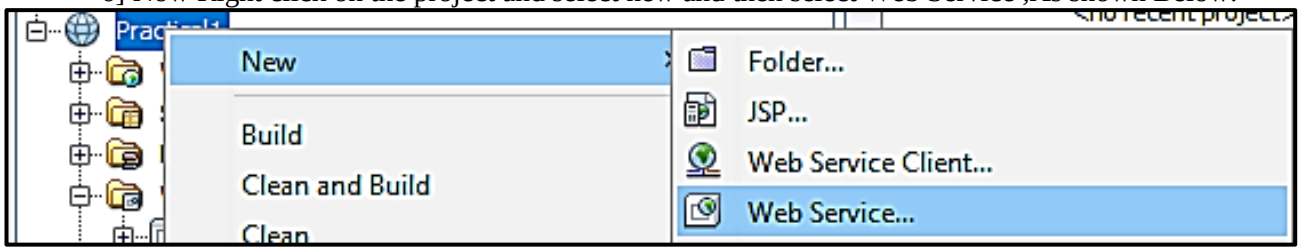
4] Now Give name to the Project Name, and click on next. You will get the following window then again, click on finish:



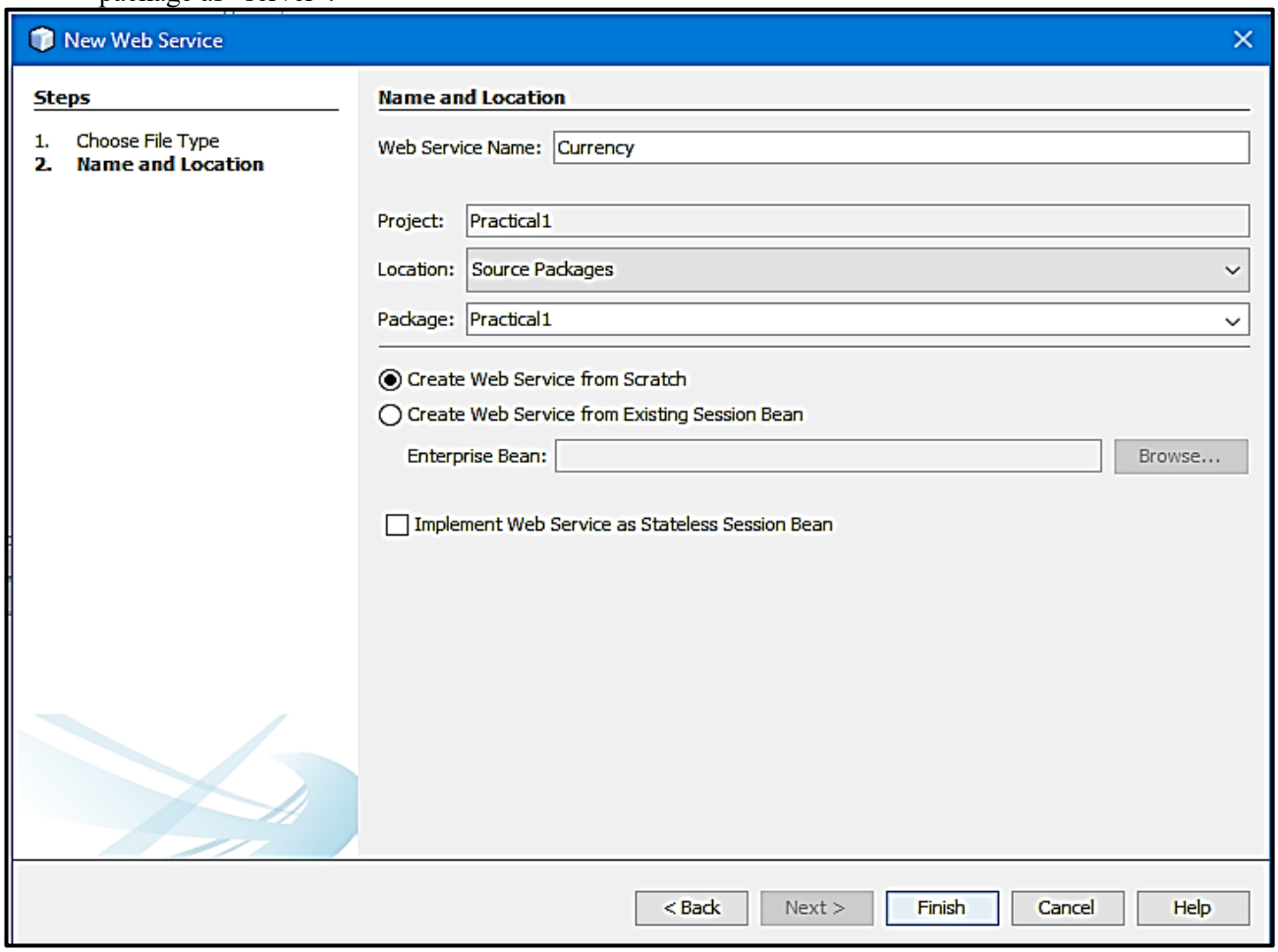
Don’t click on any frameworks, Finish.



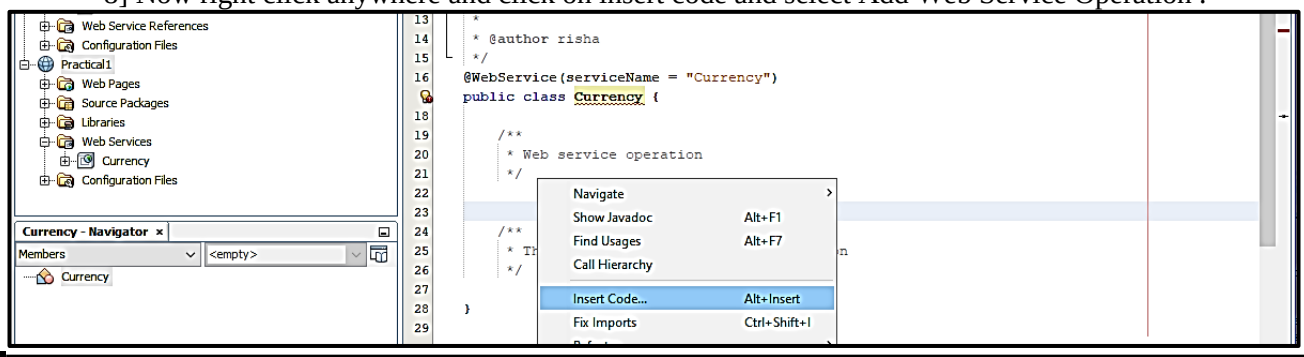
6] Now Right click on the project and select new and then select Web Service ,As shown Below:



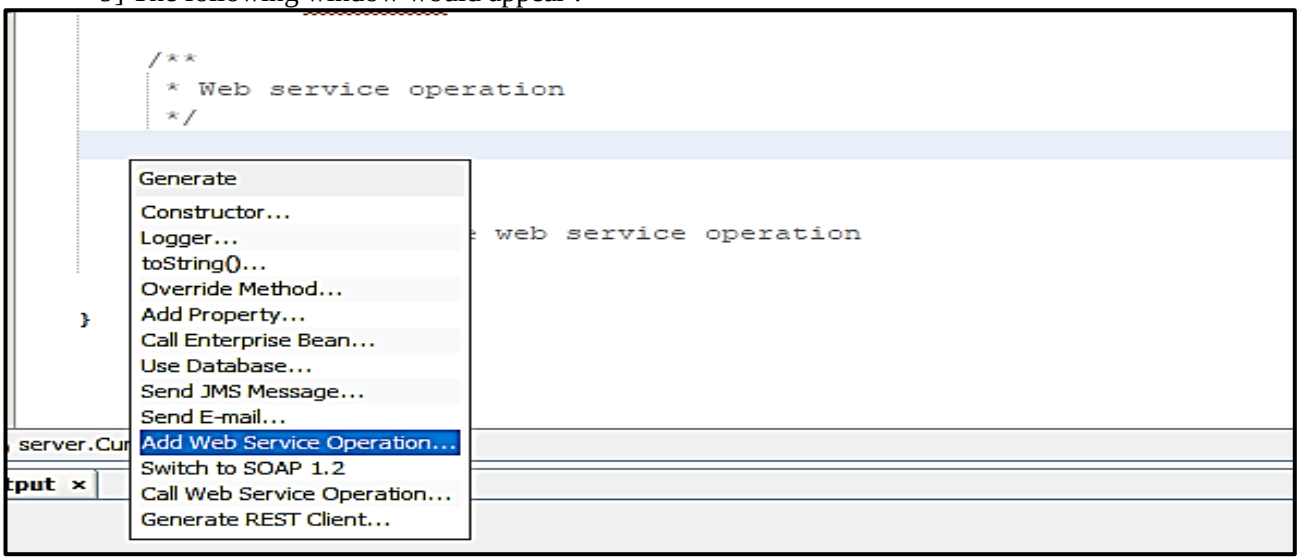
7] After clicking Web Service following window should appear , now give name to web service and give package as “server”.



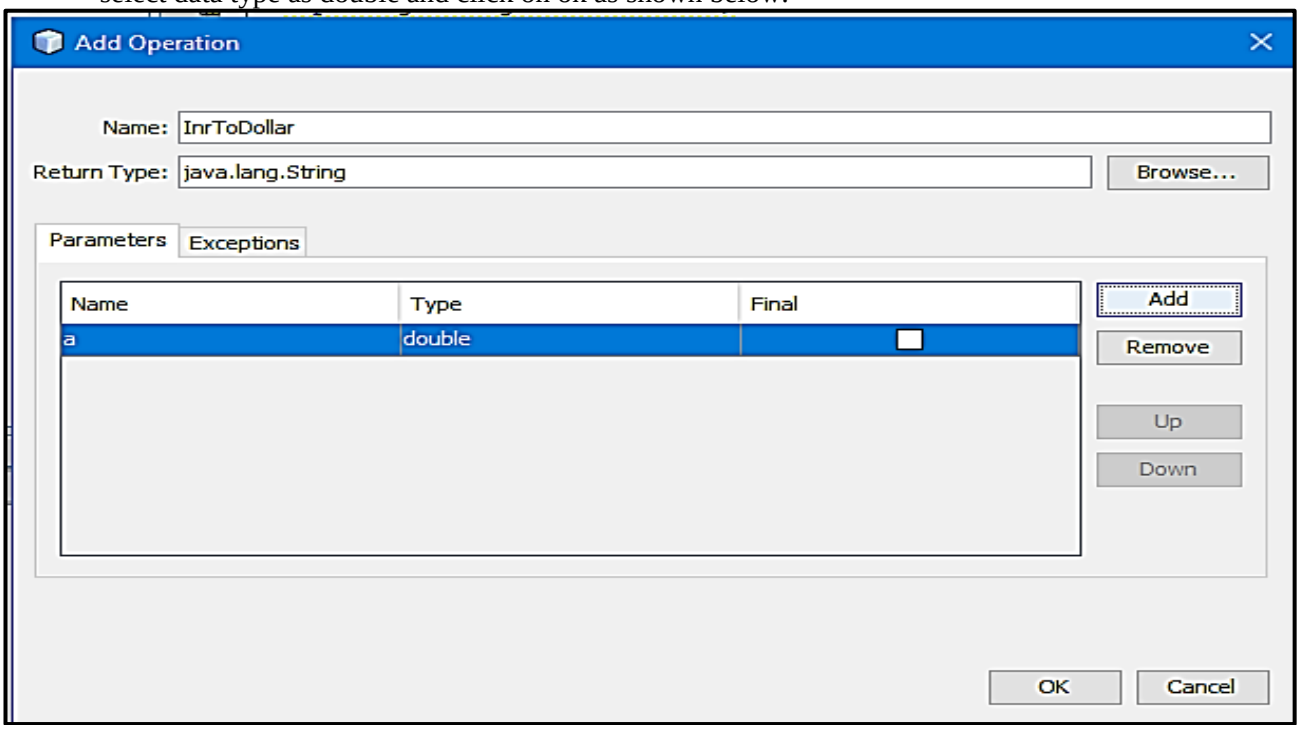
8] Now right click anywhere and click on insert code and select Add Web Service Operation :



9] The following window would appear :



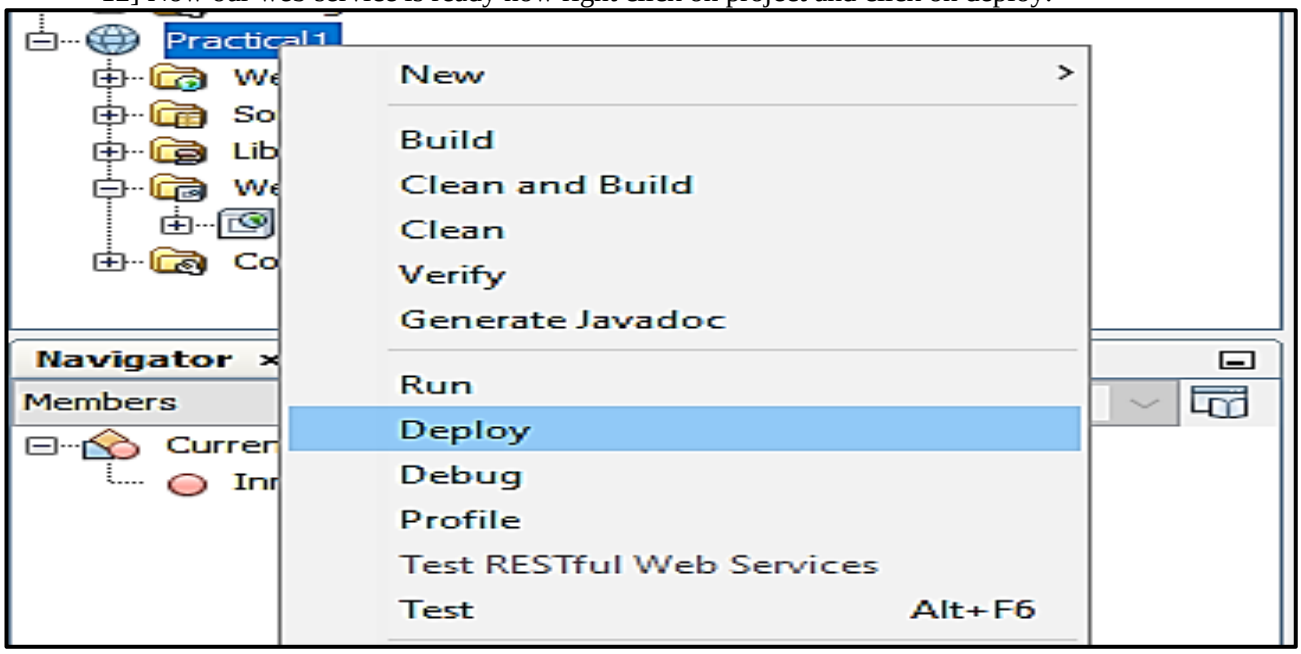
10] Just give name to the method or operation and click on add button to add parameters to the method as here we are converting dollar to rupees we should need only one parameter. give the name to the variable select data type as double and click on ok as shown below:



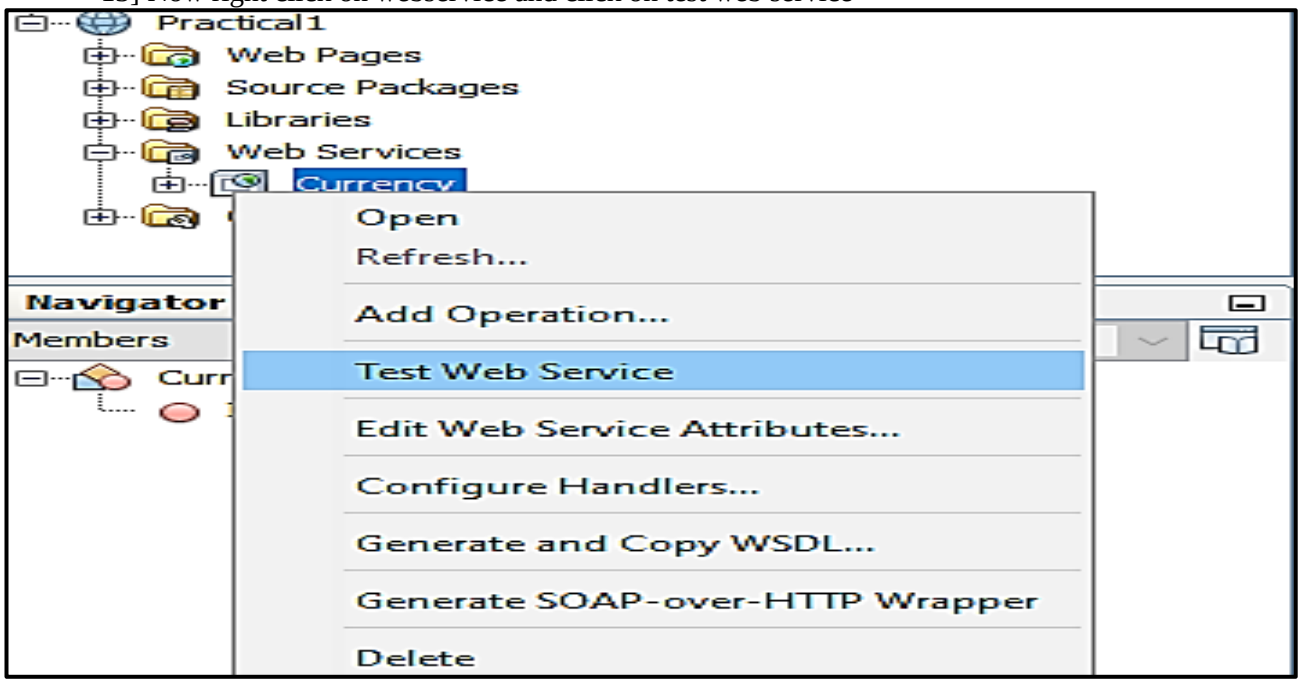
11] After clicking ok code will autogenerate, make changes in that code as mentioned below:

```
package server;

import javax.jws.WebService;
import javax.jws.WebMethod;
import javax.jws.WebParam;

@WebService(serviceName = "Currency")
public class Currency {

    @WebMethod(operationName = "InrToDollar")
    public String InrToDollar(@WebParam(name = "a") double a) {
        return "The Indian Rupees "+a+" in Dollars is"+(a/91.56);
    }

}
```

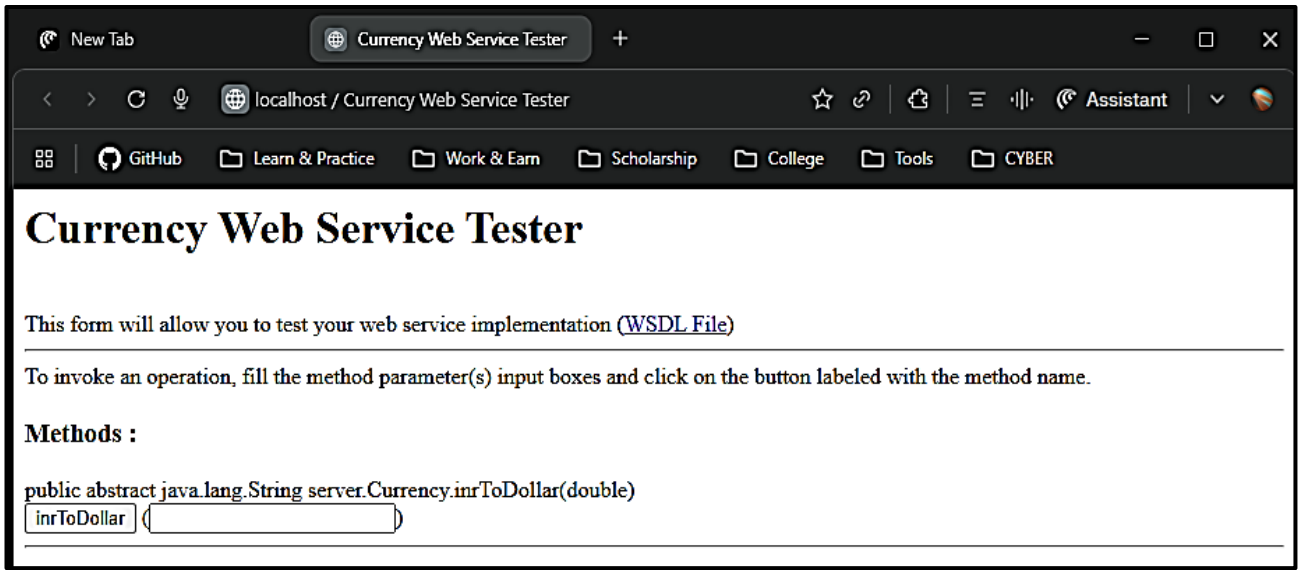
12] Now our web service is ready now right click on project and click on deploy:



13] Now right click on webservice and click on test web service



Output:



New Tab | Method invocation trace

localhost / Method invocation trace

GitHub | Learn & Practice | Work & Earn | Scholarship | College | Tools | CYBER

inrtoDollar Method invocation

Method parameter(s)

Type	Value
double	1233

Method returned

java.lang.String : "**The Indian Rupees 1233.0 in Dollar is 13.643908376673675**"

SOAP Request


```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Header/>
    <S:Body>
        <ns2:InrtoDollar xmlns:ns2="http://server/">
            <a>1233.0</a>
        </ns2:InrtoDollar>
    </S:Body>
</S:Envelope>
```


SOAP Response


```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Header/>
    <S:Body>
        <ns2:InrtoDollarResponse xmlns:ns2="http://server/">
            <return>The Indian Rupees 1233.0 in Dollar is 13.643908376673675</return>
        </ns2:InrtoDollarResponse>
    </S:Body>
</S:Envelope>
```

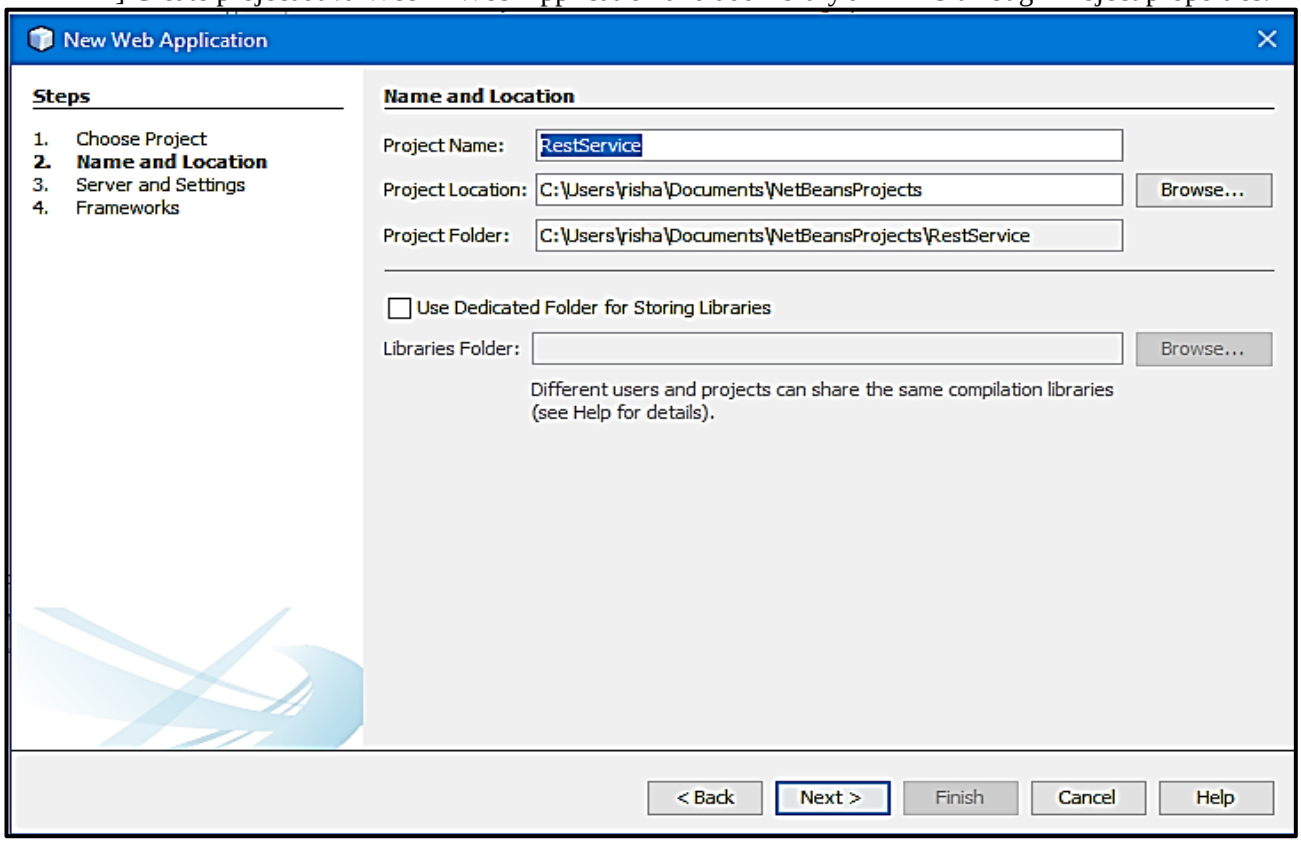

So this is how we created our web service and deployed it

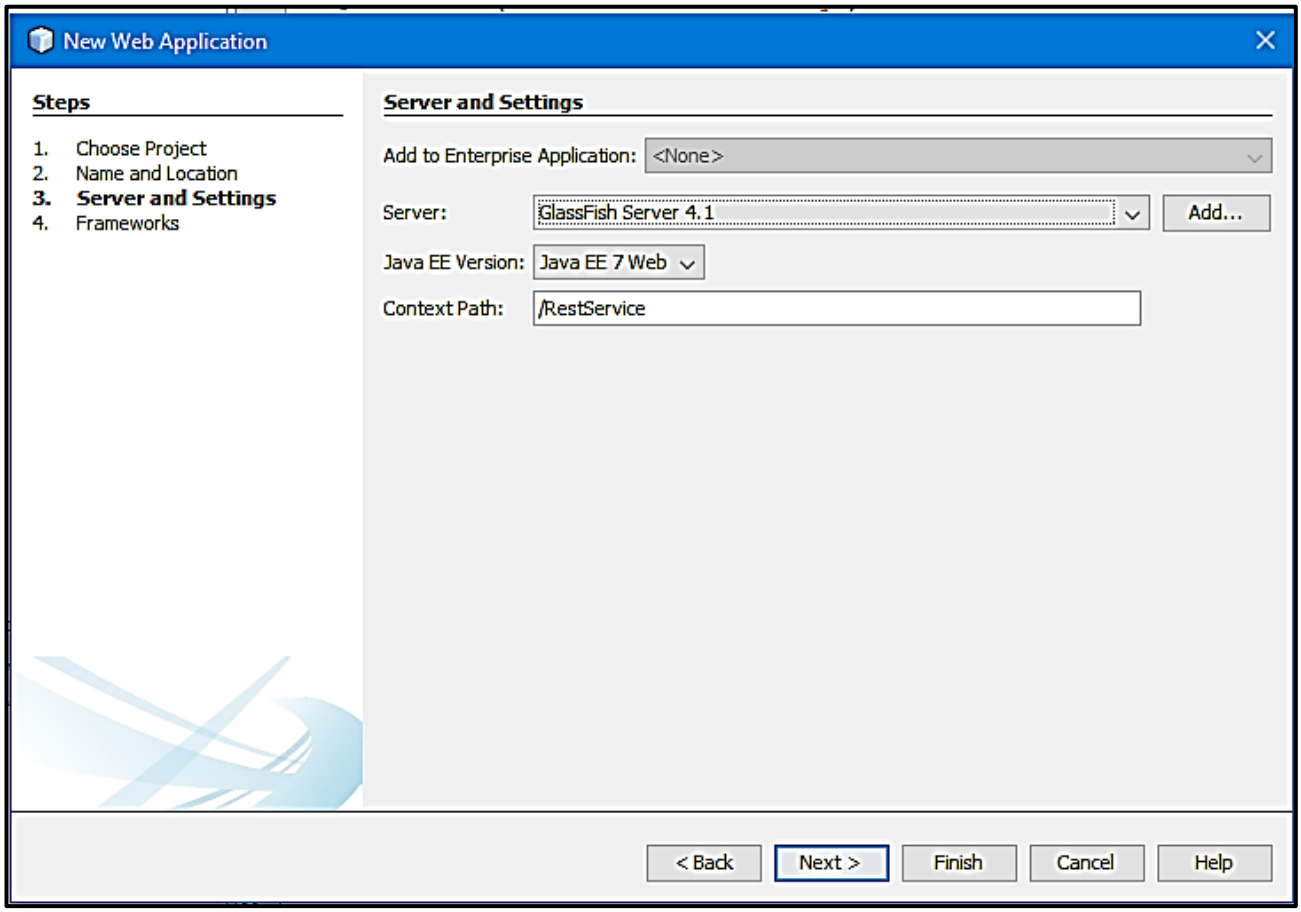
Practical 3

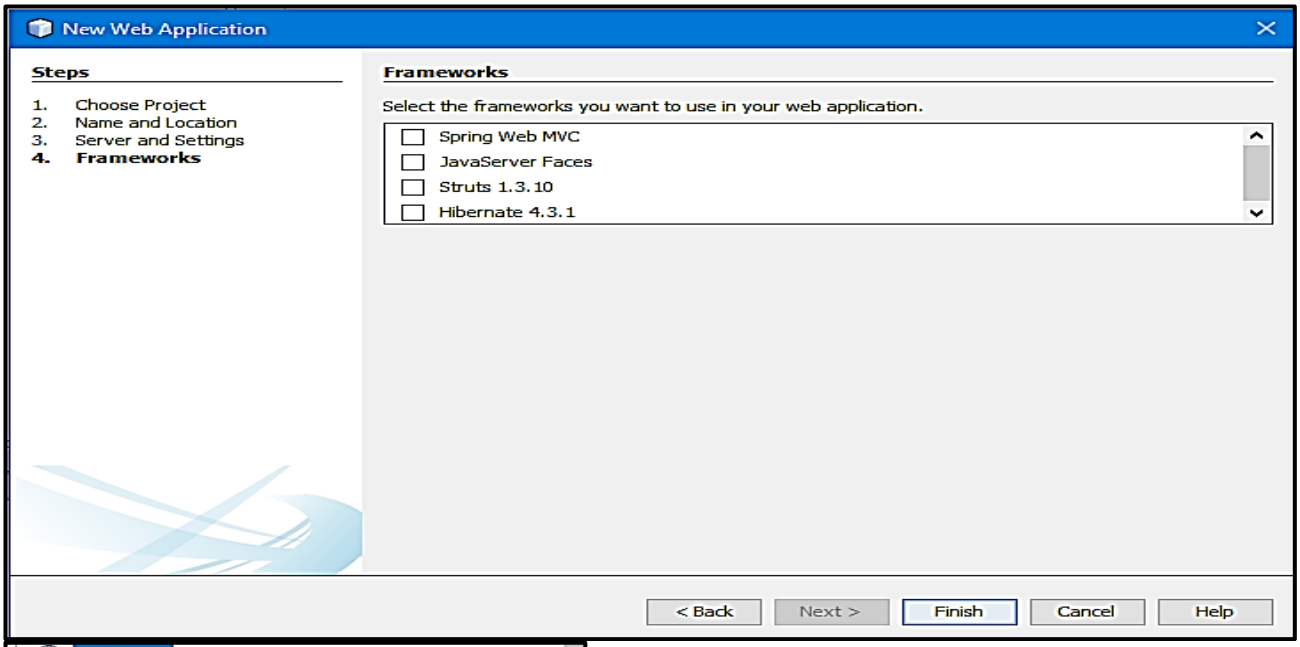
Aim: Create a Simple REST Service.

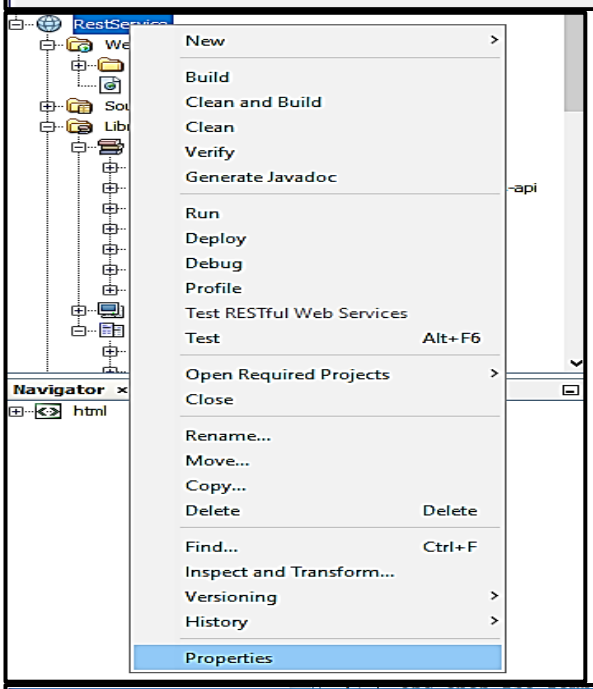
Steps:

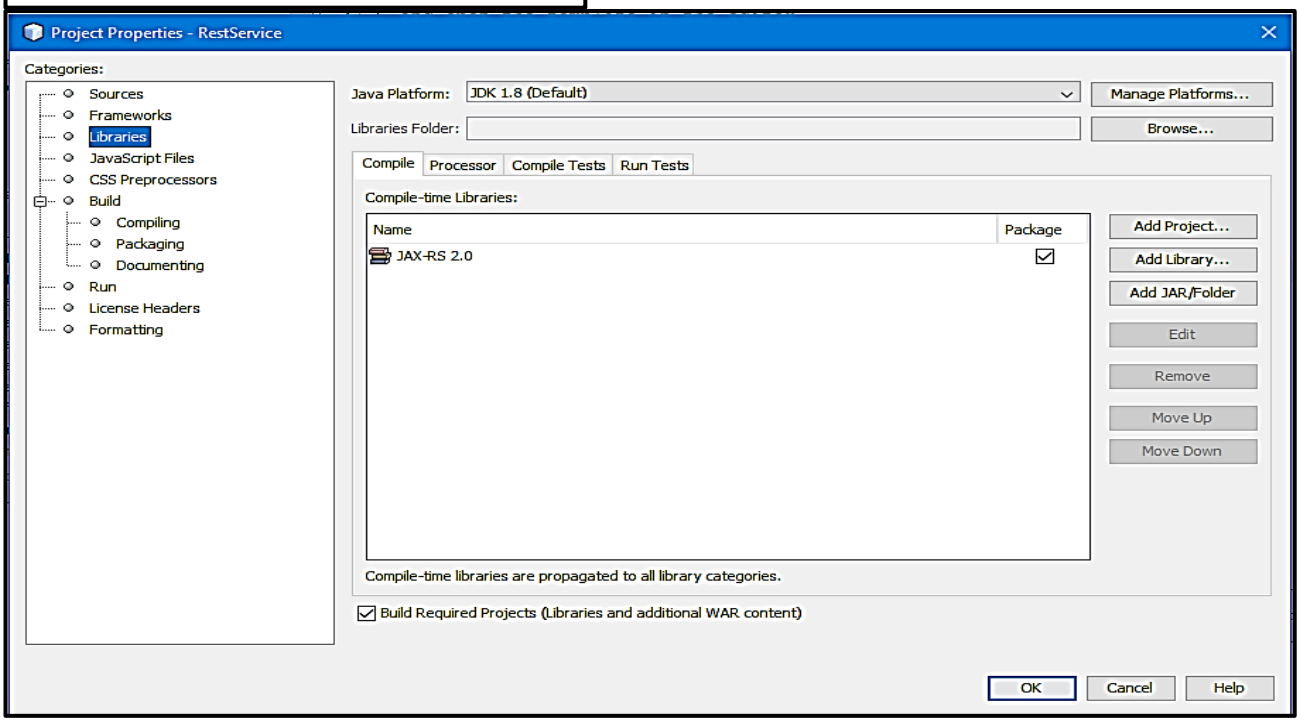
1] Create project Java Web →Web Application and add library JAX-RS through Project properties.



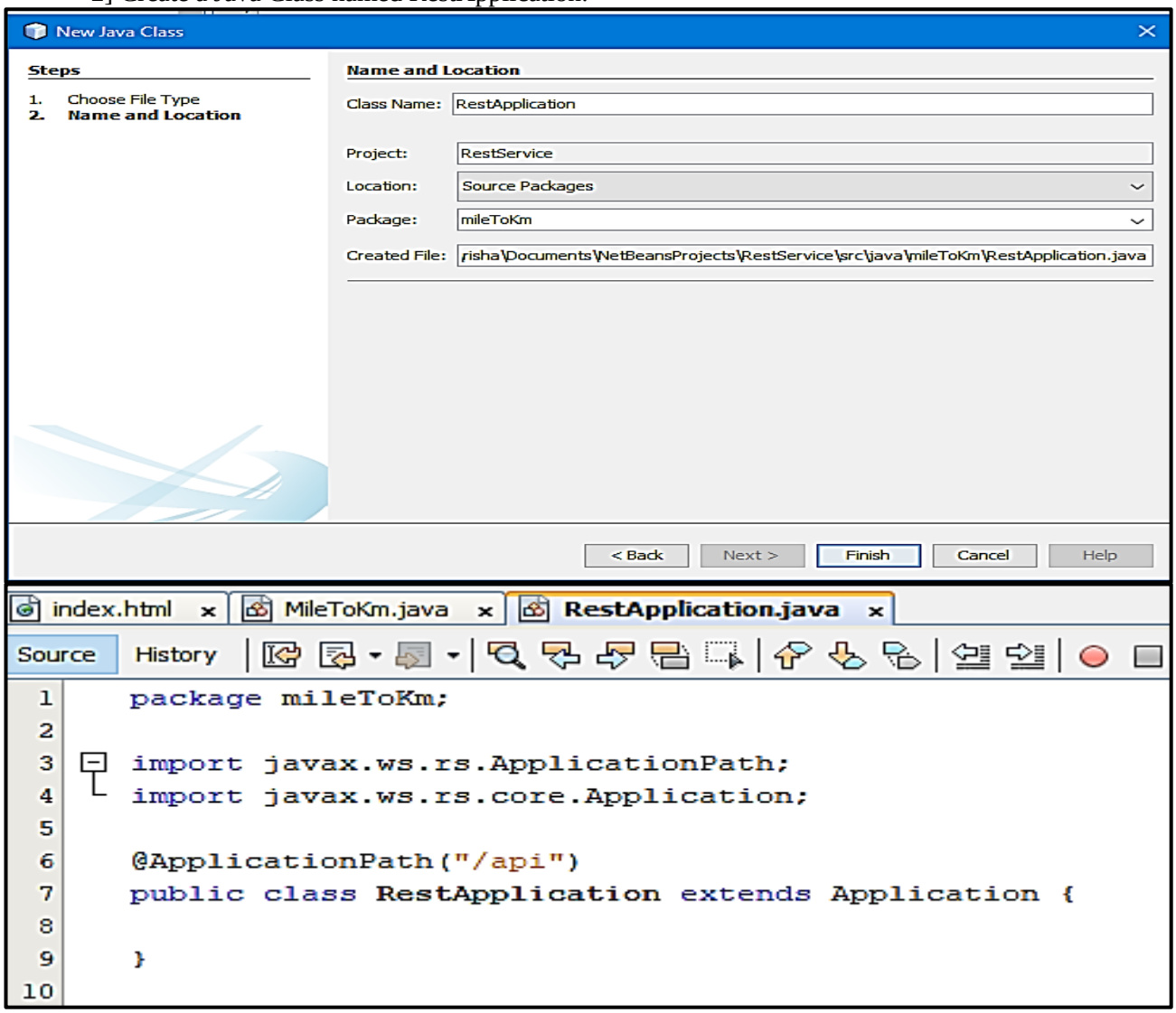



New Web Application
Steps
1. Choose Project
2. Name and Location
3. Server and Settings
4. Frameworks
Frameworks
Select the frameworks you want to use in your web application.
Spring Web MVC
JavaServer Faces
Struts 1.3.10
Hibernate 4.3.1
< Back
Next >
Finish
Cancel
Help


RestService
New
Build
Clean and Build
Clean
Verify
Generate Javadoc
Run
Deploy
Debug
Profile
Test RESTful Web Services
Test Alt+F6
Open Required Projects
Close
Rename...
Move...
Copy...
Delete Delete
Find... Ctrl+F
Inspect and Transform...
Versioning
History
Properties
Navigator
html


Project Properties - RestService
Categories:
Sources
Frameworks
Libraries
JavaScript Files
CSS Preprocessors
Build
Compiling
Packaging
Documenting
Run
License Headers
Formatting
Java Platform: JDK 1.8 (Default)
Manage Platforms...
Libraries Folder:
Browse...
Compile
Processor
Compile Tests
Run Tests
Compile-time Libraries:
Name
Package
JAX-RS 2.0
Add Project...
Add Library...
Add JAR/Folder
Edit
Remove
Move Up
Move Down
Compile-time libraries are propagated to all library categories.
Build Required Projects (Libraries and additional WAR content)
OK
Cancel
Help

2] Create a Java Class named RestApplication.




```
package mileToKm;

import javax.ws.rs.ApplicationPath;
import javax.ws.rs.core.Application;

@ApplicationPath("/api")
public class RestApplication extends Application {

}
```

3] Then create another Java class named MileToKmRestService.

```
package MileToKmRestService;

import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.QueryParam;

@Path("/converter") // This sets the path for this service to /api/converter
public class MileToKm {
    @GET
    @Path("/mileToKm") // This sets the path for this specific method
    public String convertMileToKm(@QueryParam("miles") double miles) {
        double km = miles * 1.60934; // Conversion logic: 1 mile = 1.60934 km
        return miles + " miles = " + km + " kilometers";
    }
}
```

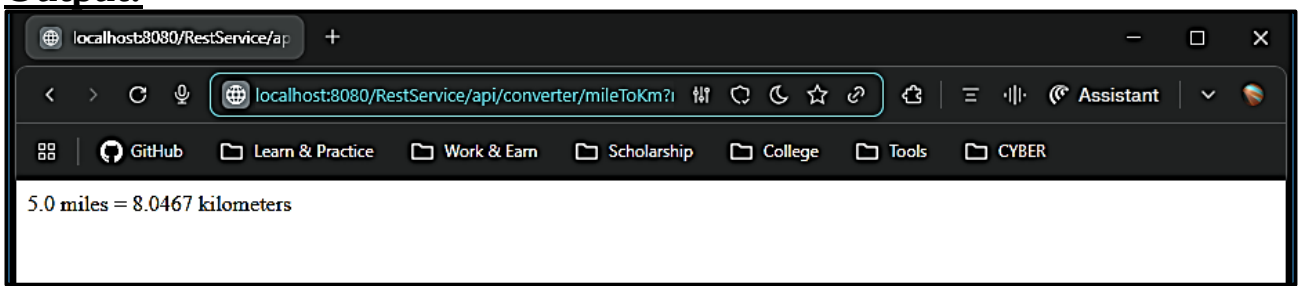
index.html | MileToKm.java | RestApplication.java

Source | History

```java
package MileToKmRestService;

import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.QueryParam;

@Path("/converter") // This sets the path for this service to /api/converter
public class MileToKm {
    @GET
    @Path("/mileToKm") // This sets the path for this specific method
    public String convertMileToKm(@QueryParam("miles") double miles) {
        double km = miles * 1.60934; // Conversion logic: 1 mile = 1.60934 km
        return miles + " miles = " + km + " kilometers";
    }
}
```

4] Now Deploy project and Go to → http://localhost:8080/RestService/api/converter/mileToKm?miles=5

Output:

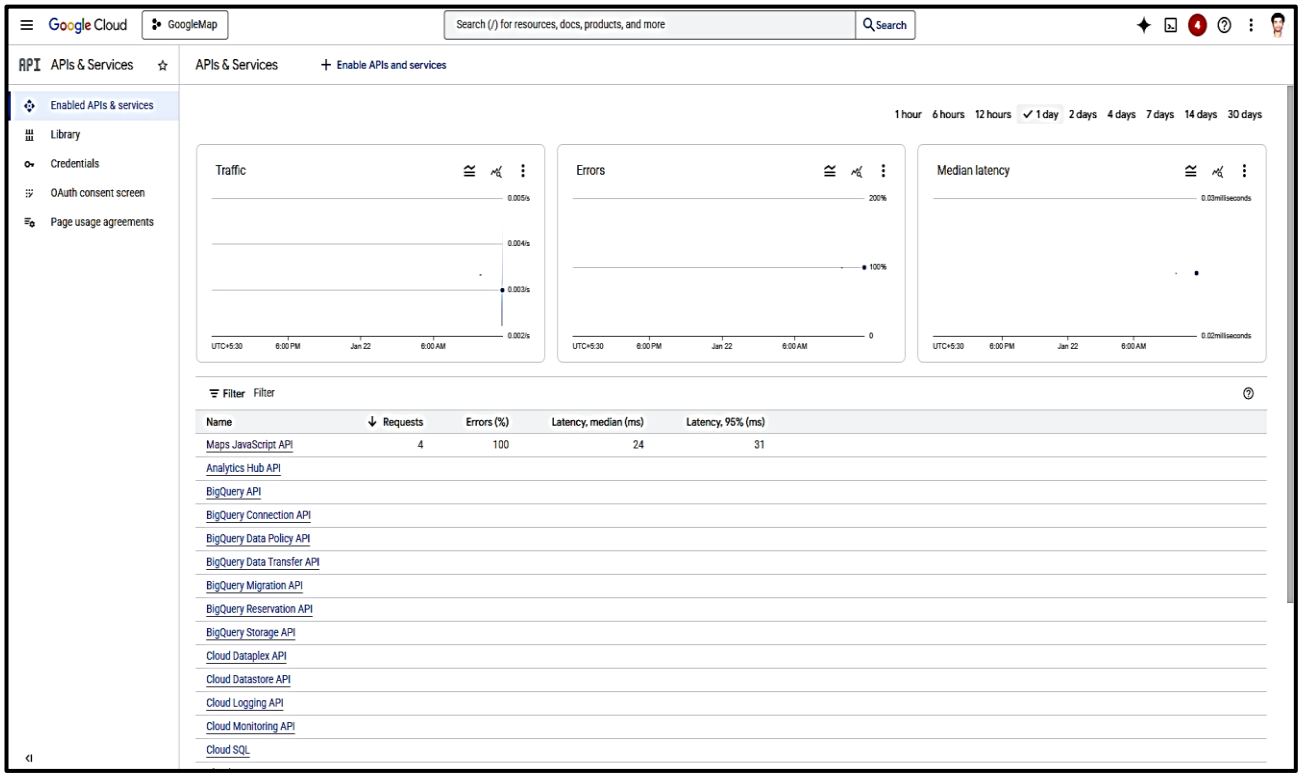


Practical 4

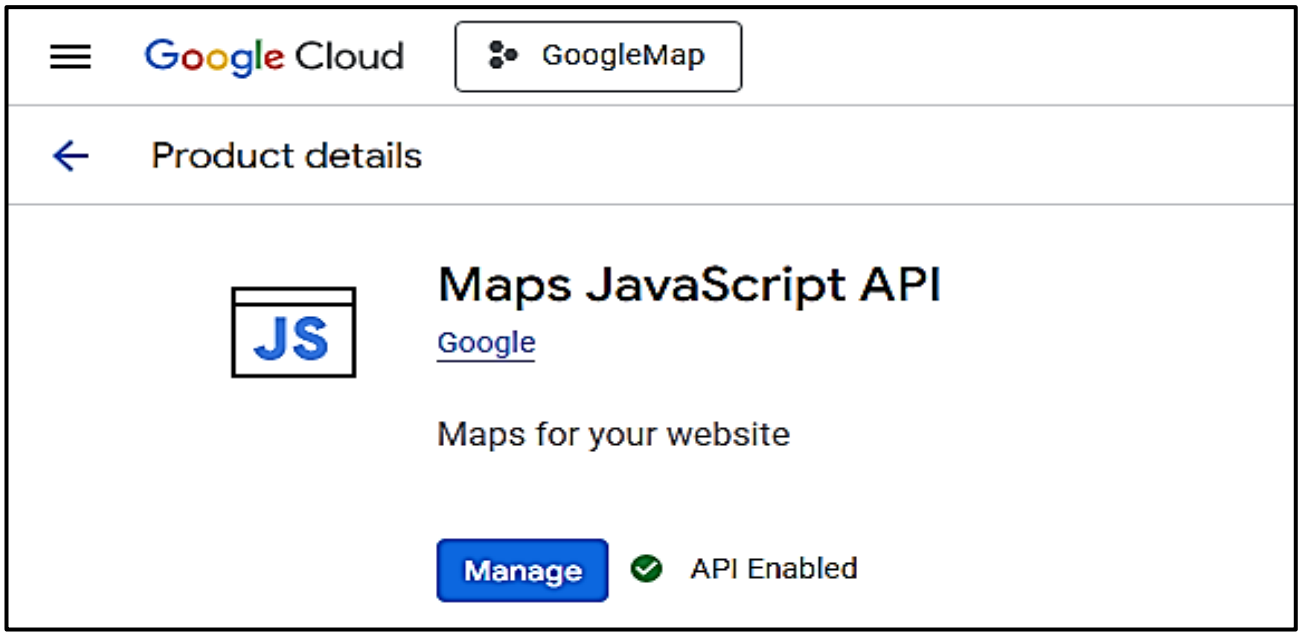
<u>Aim:</u> Develop application to consume Google's search / Google's Map RESTful Web service.

<u>Steps:</u>

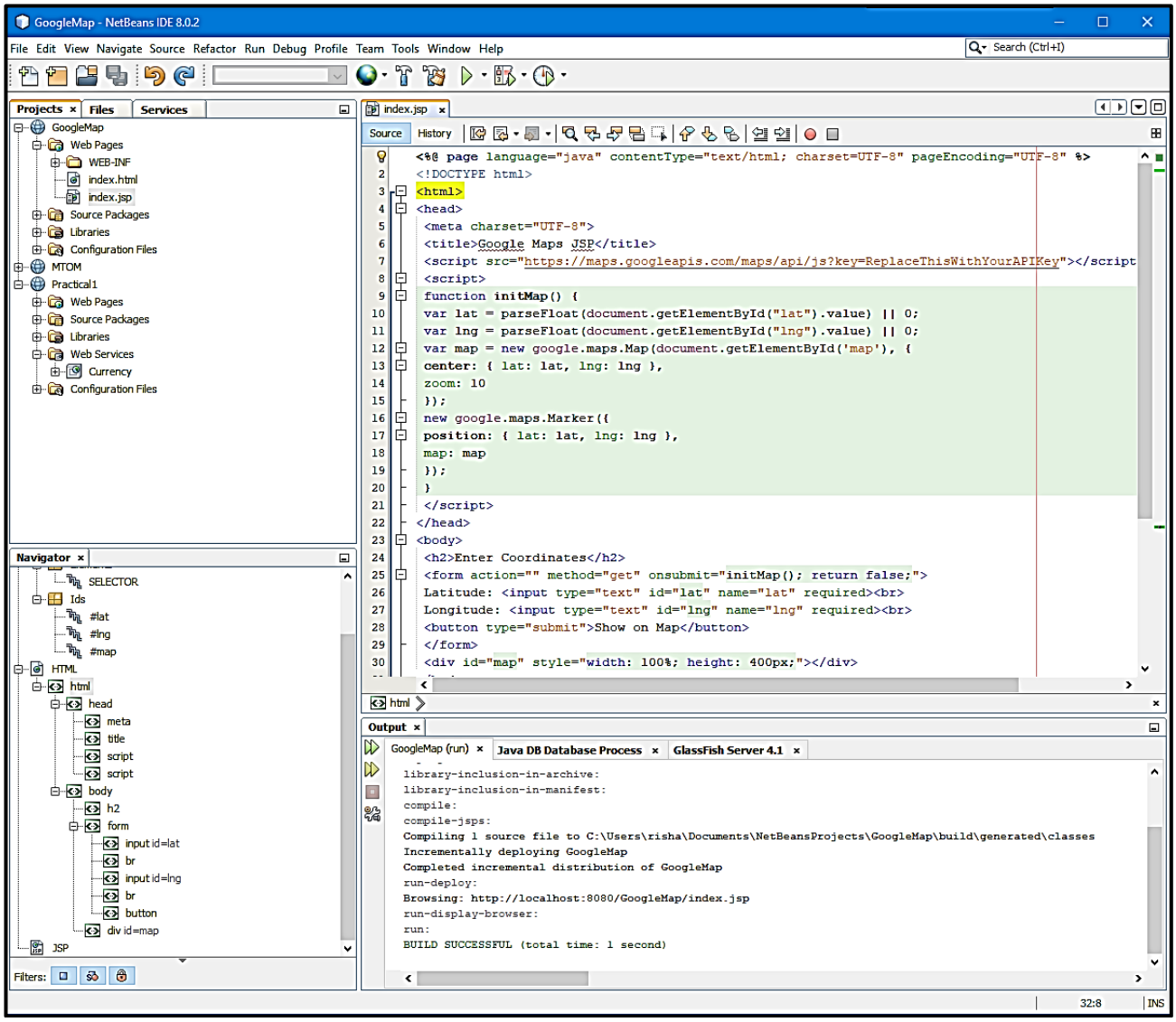
1] Visit console.cloud.google.com, go to APIs and services and select Enabled APIs and services.



2] Enable the following API :



3] Now create a Web Application in NetBeans and type the following in index.jsp file.

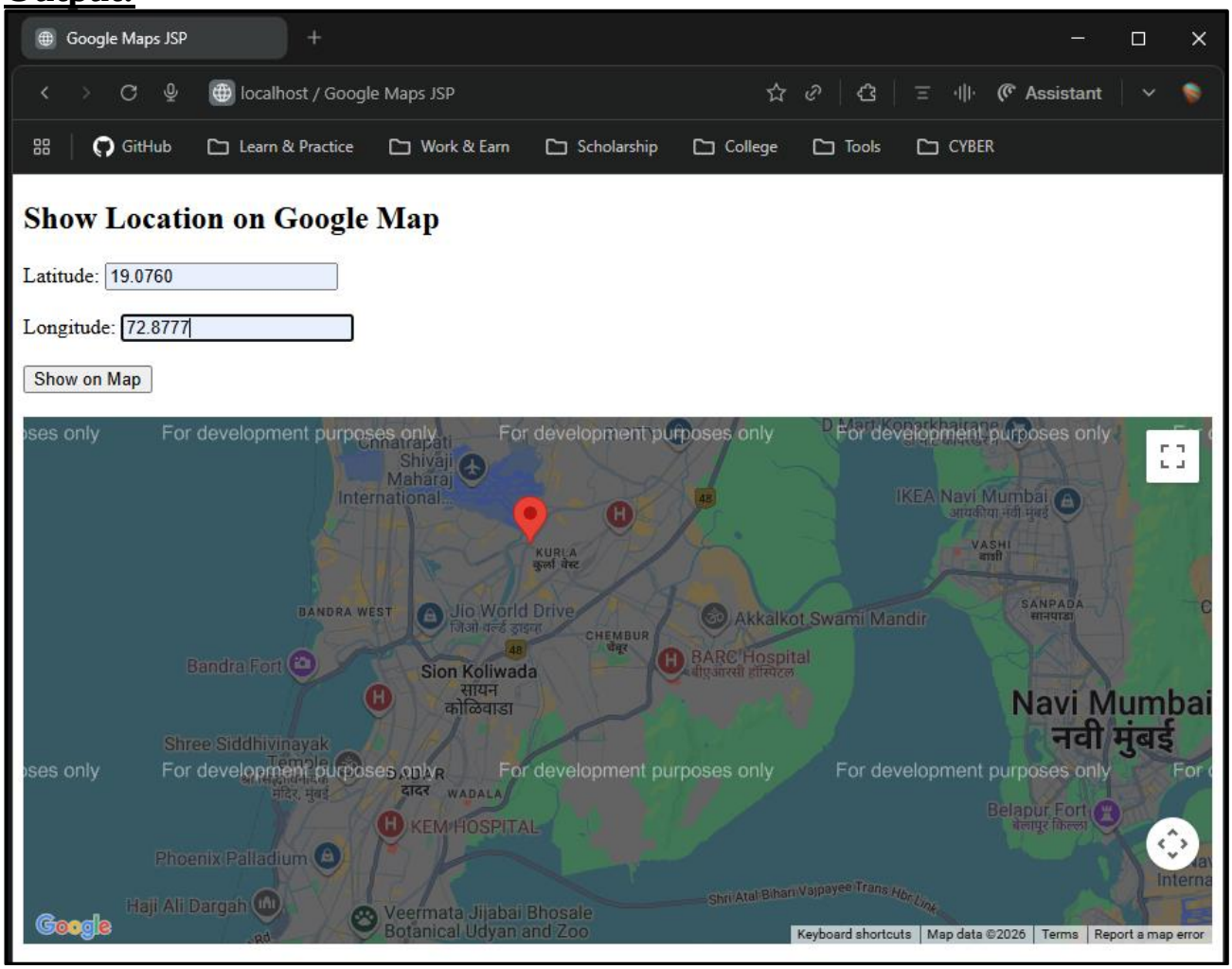



```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Google Maps JSP</title>
<script src="https://maps.googleapis.com/maps/api/js?key=ReplaceThisWithYourAPIKey"></script>
<script>
 function initMap() {
 var lat = parseFloat(document.getElementById("lat").value) || 0;
 var lng = parseFloat(document.getElementById("lng").value) || 0;
 var map = new google.maps.Map(document.getElementById('map'), {
 center: { lat: lat, lng: lng },
 zoom: 10
 });
 new google.maps.Marker({
 position: { lat: lat, lng: lng },
 map: map
 });
 }
</script>
</head>
<body>
<h2>Enter Coordinates</h2>
<form action="" method="get" onsubmit="initMap(); return false;">
 Latitude: <input type="text" id="lat" name="lat" required><br>
```

```
Longitude: <input type="text" id="lng" name="lng" required><br>
<button type="submit">Show on Map</button>
</form>
<div id="map" style="width: 100%; height: 400px;"></div>
</body>
</html>
```

4] Visit the index.jsp file on browser and enter some random co-ordinates to test the API

Output:

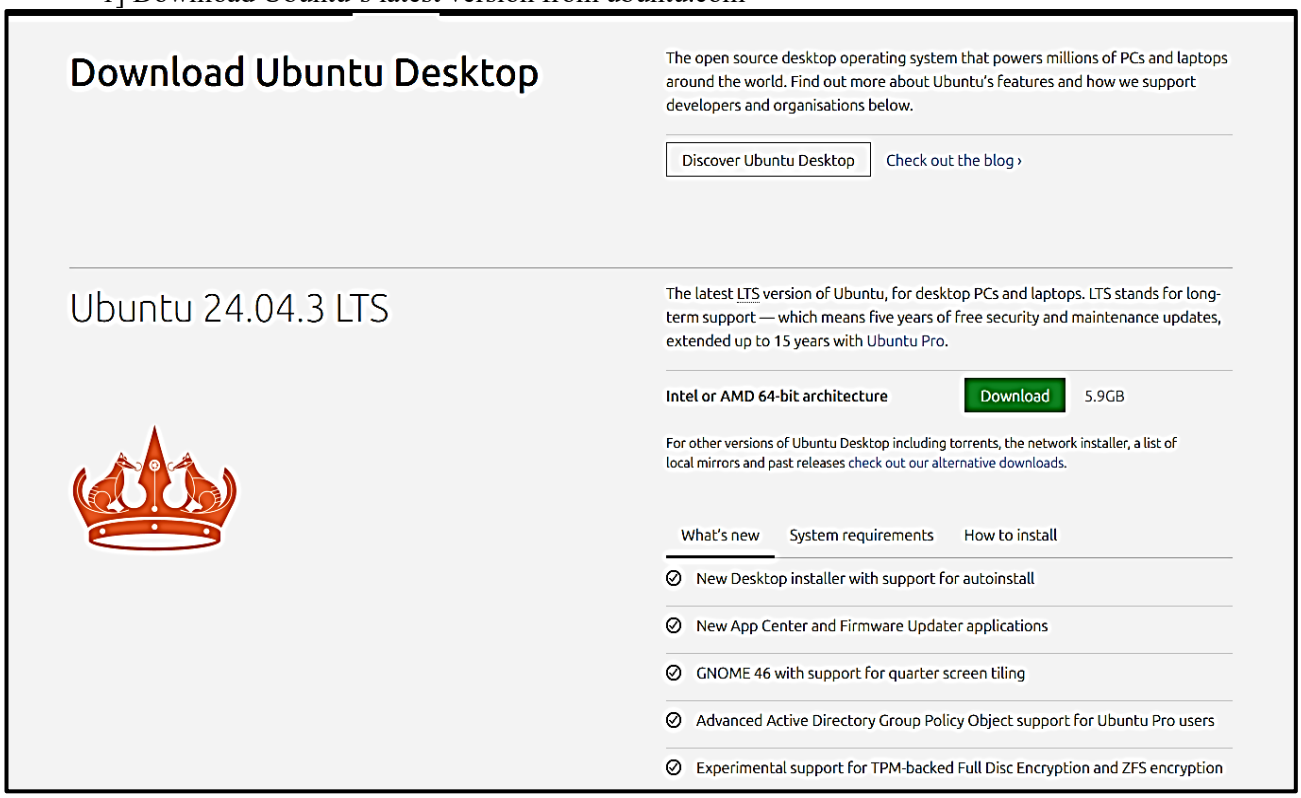


Practical 5

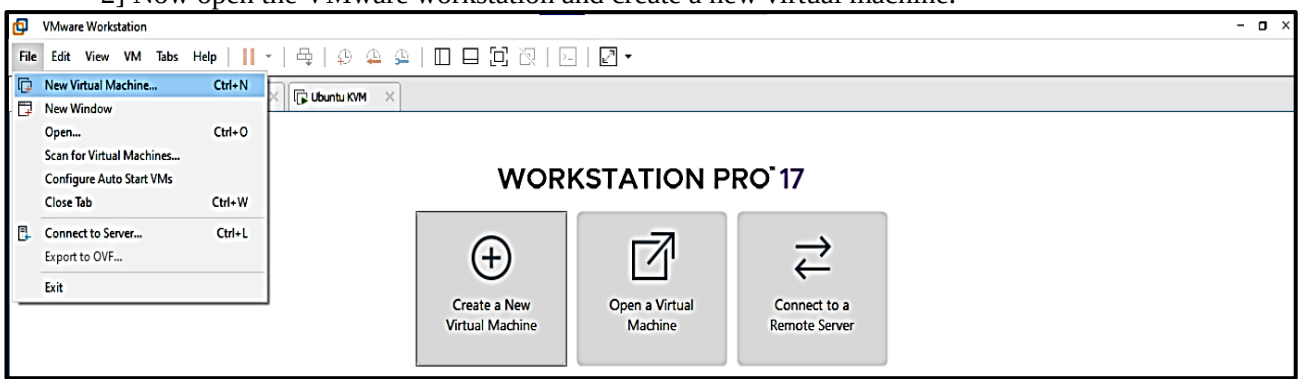
Aim: Installation and Configuration of virtualization using KVM.

Steps:

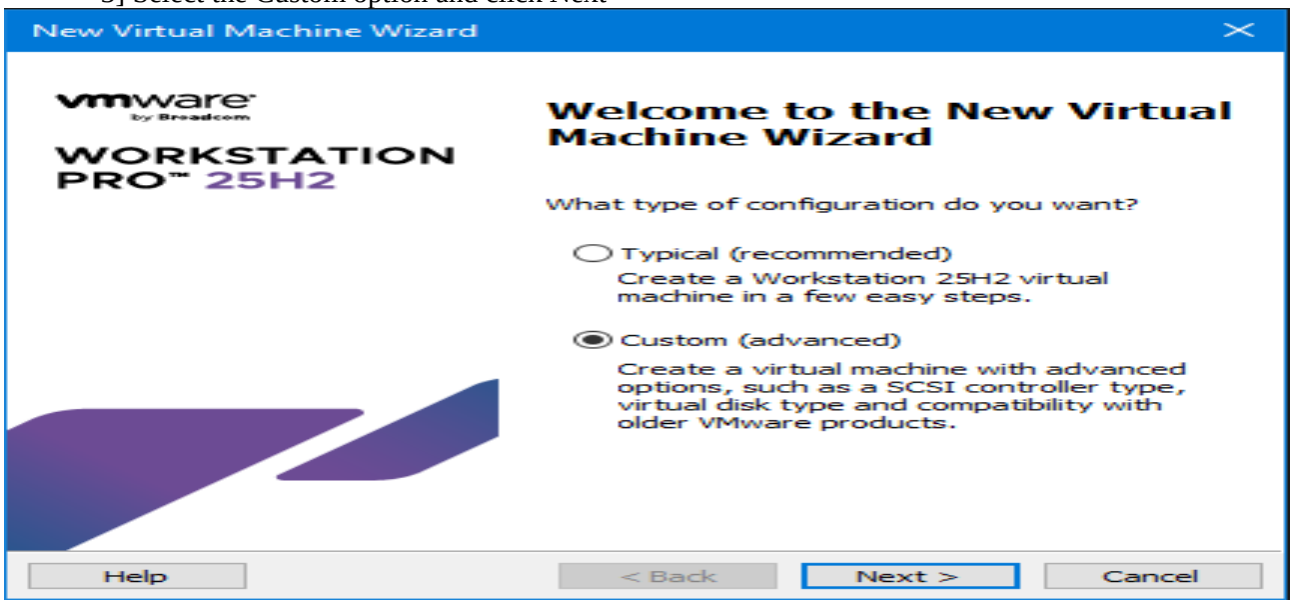
1] Download Ubuntu's latest version from ubuntu.com



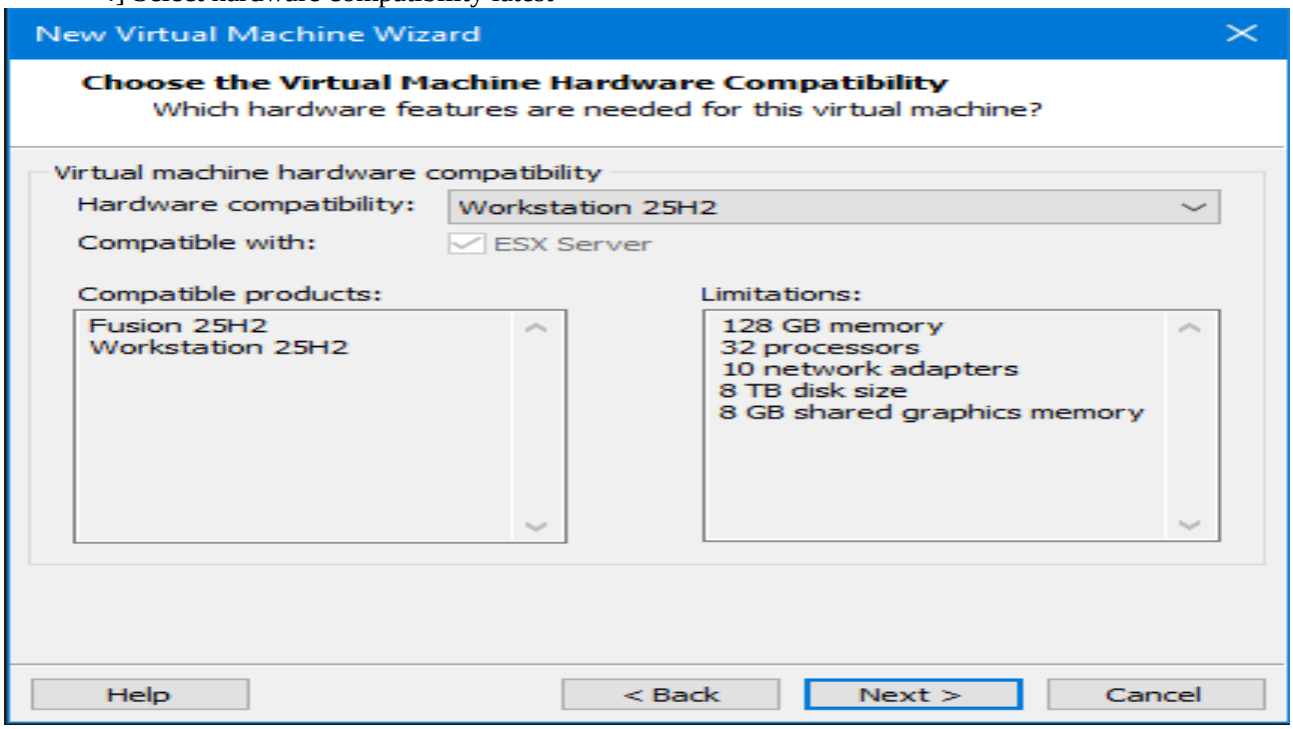
2] Now open the VMware workstation and create a new virtual machine.



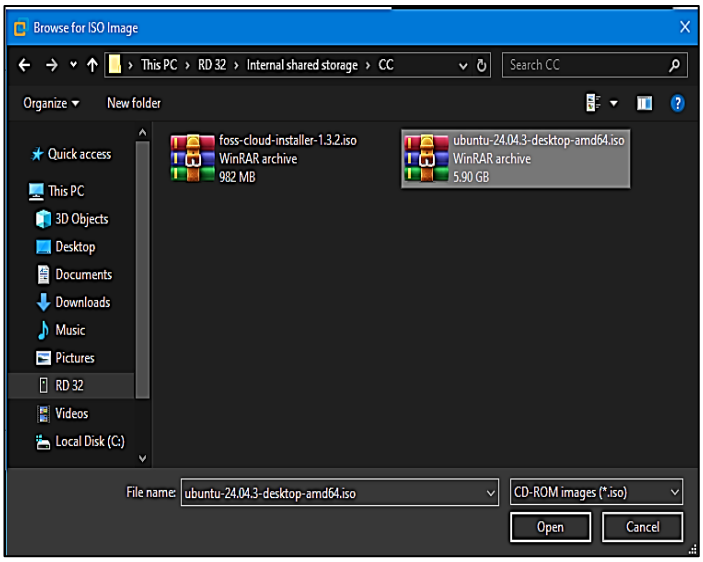
3] Select the Custom option and click Next

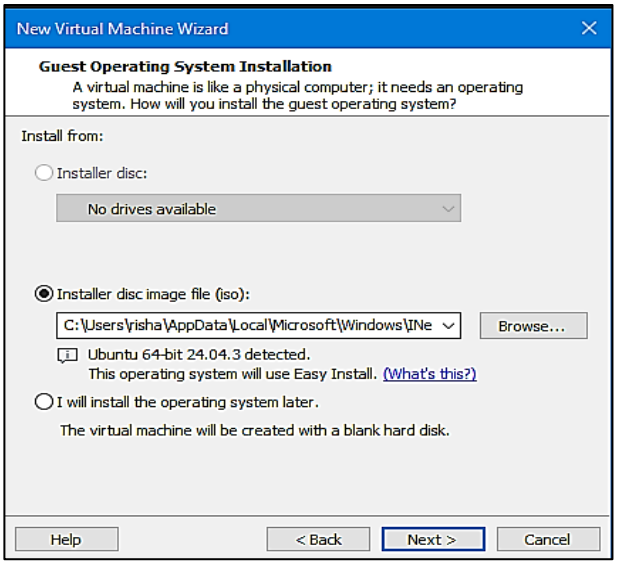


4] Select hardware compatibility latest

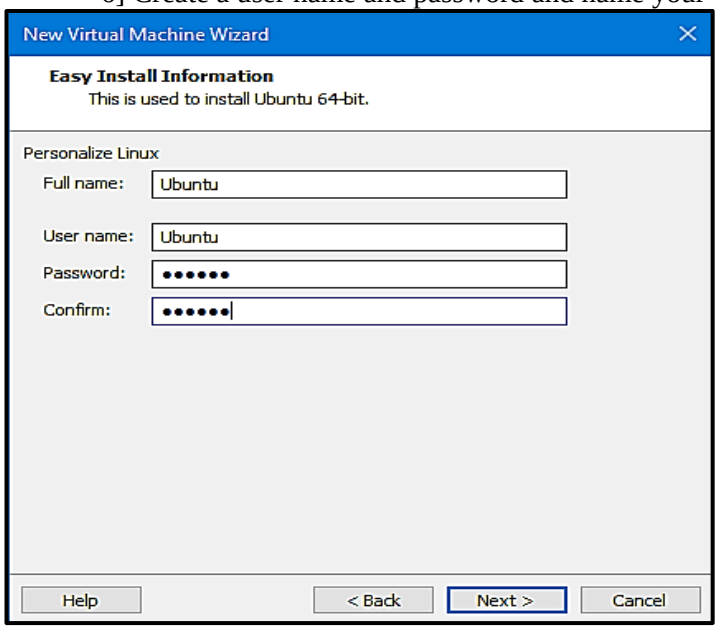


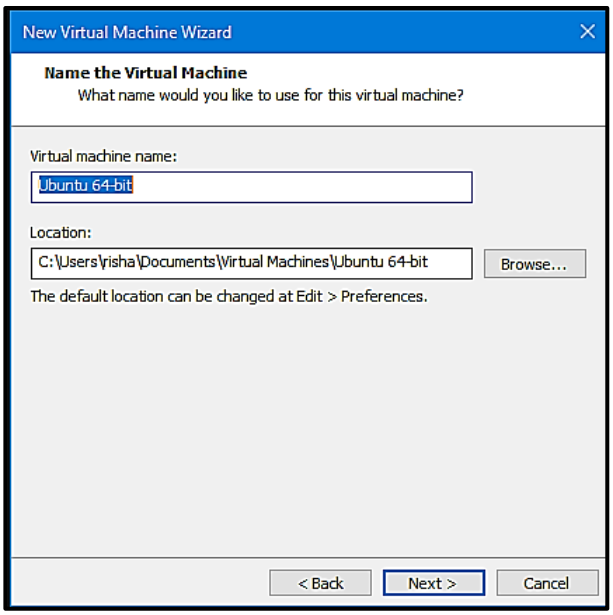
5] Browse the file in Ubuntu's file manager and select it



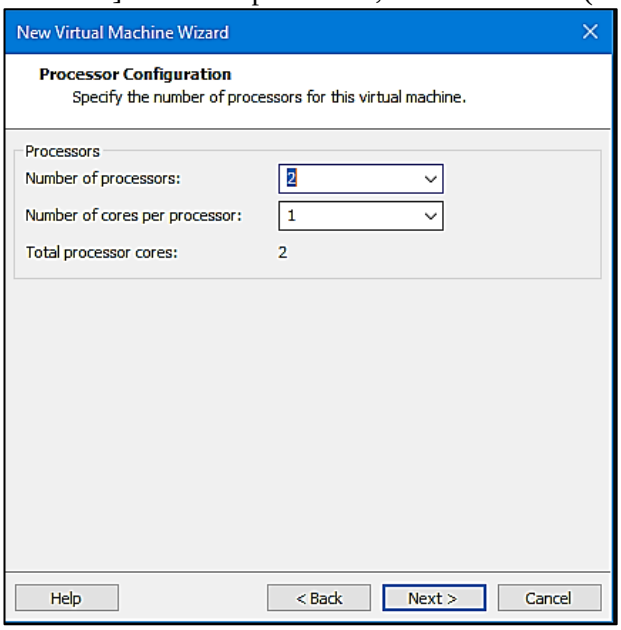


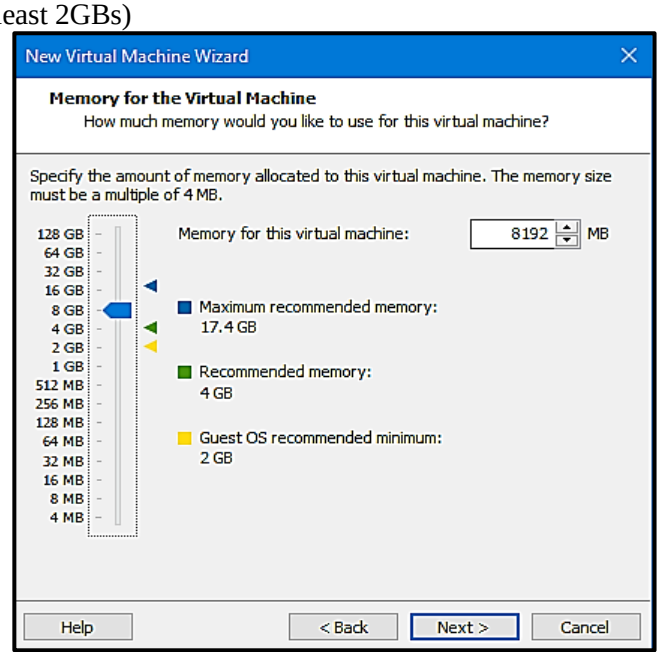
6] Create a user name and password and name your Virtual Machine.



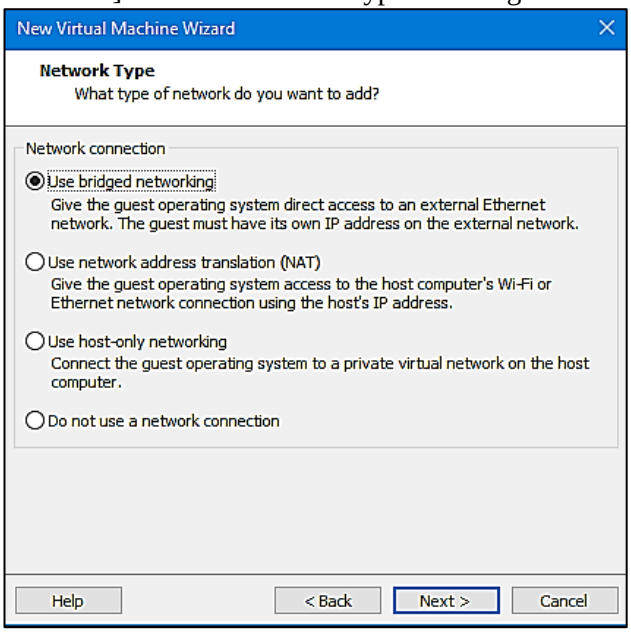


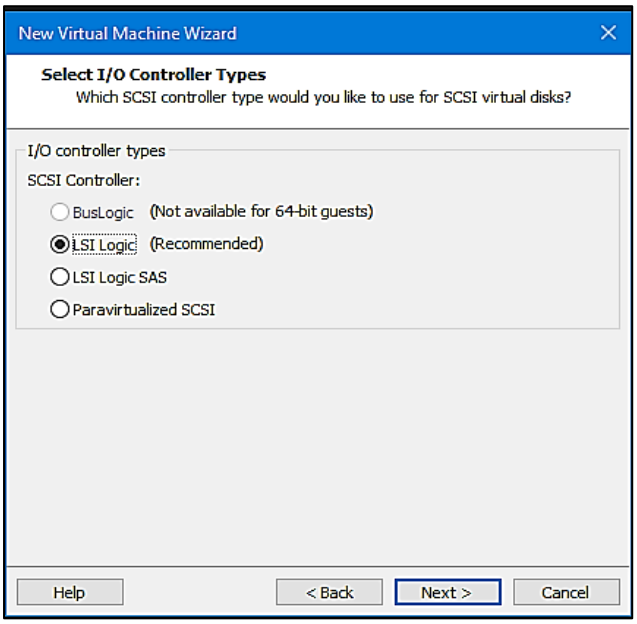
7] Select the processors, cores and RAM (atleast 2GBs)



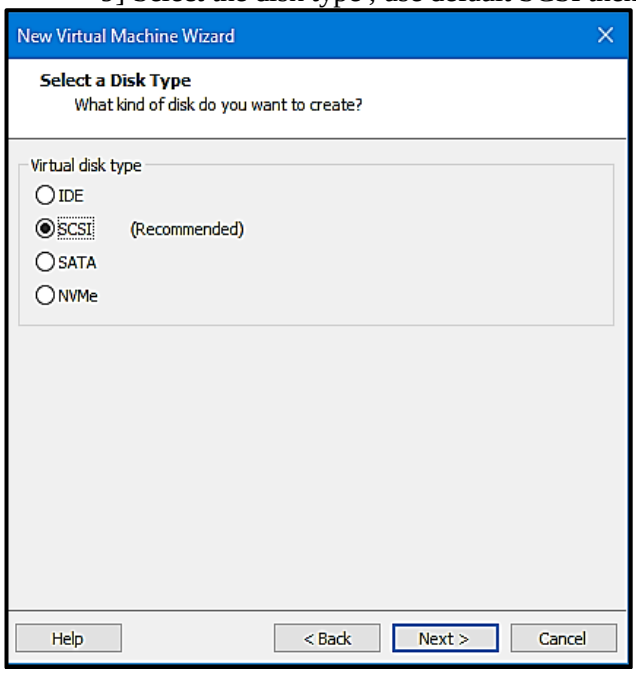


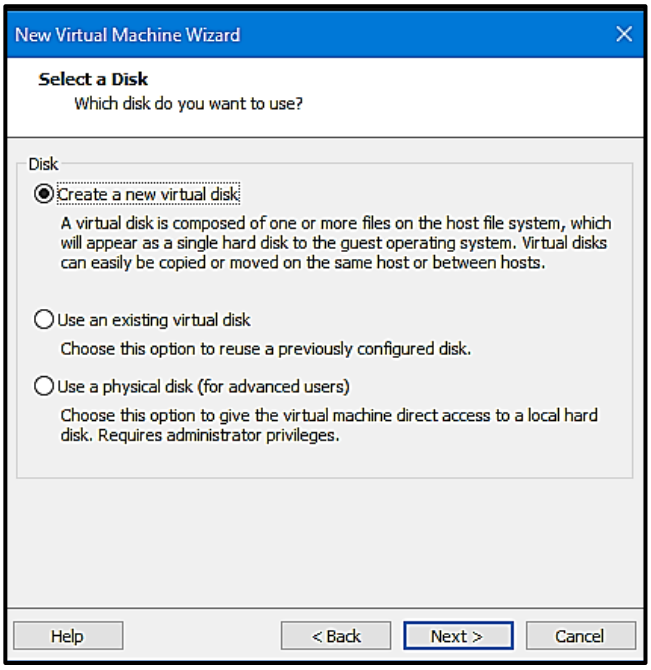
8] Select the network type use bridge network for better experience and then input /output controller



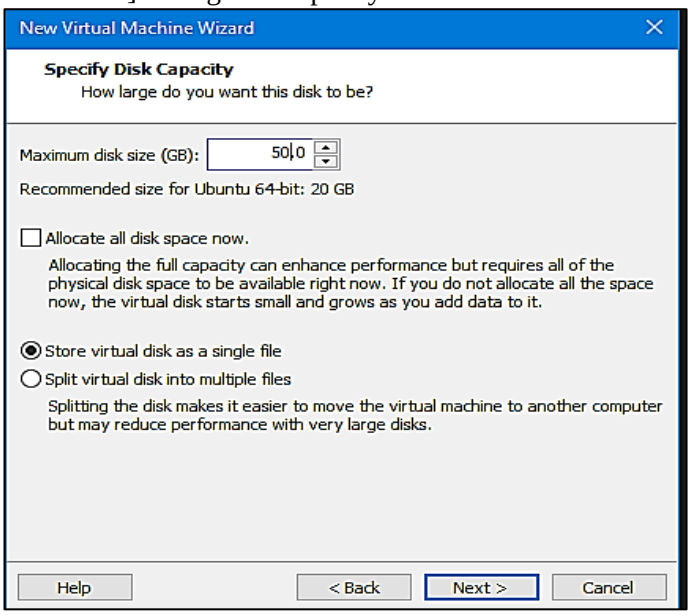


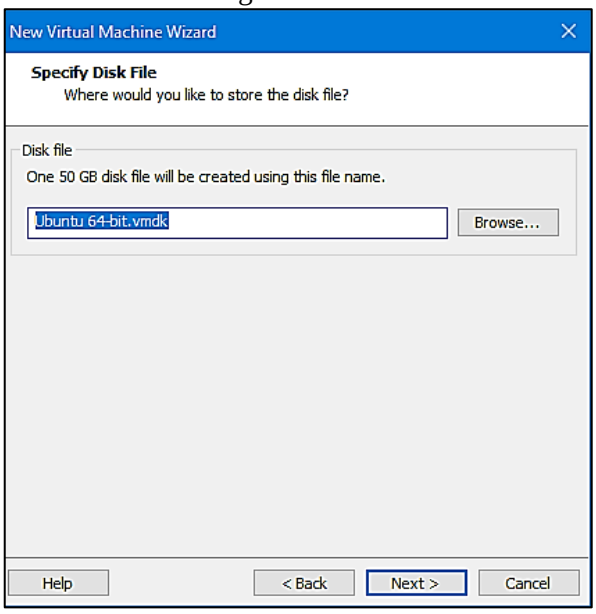
9] Select the disk type , use default SCSI then select a disk.



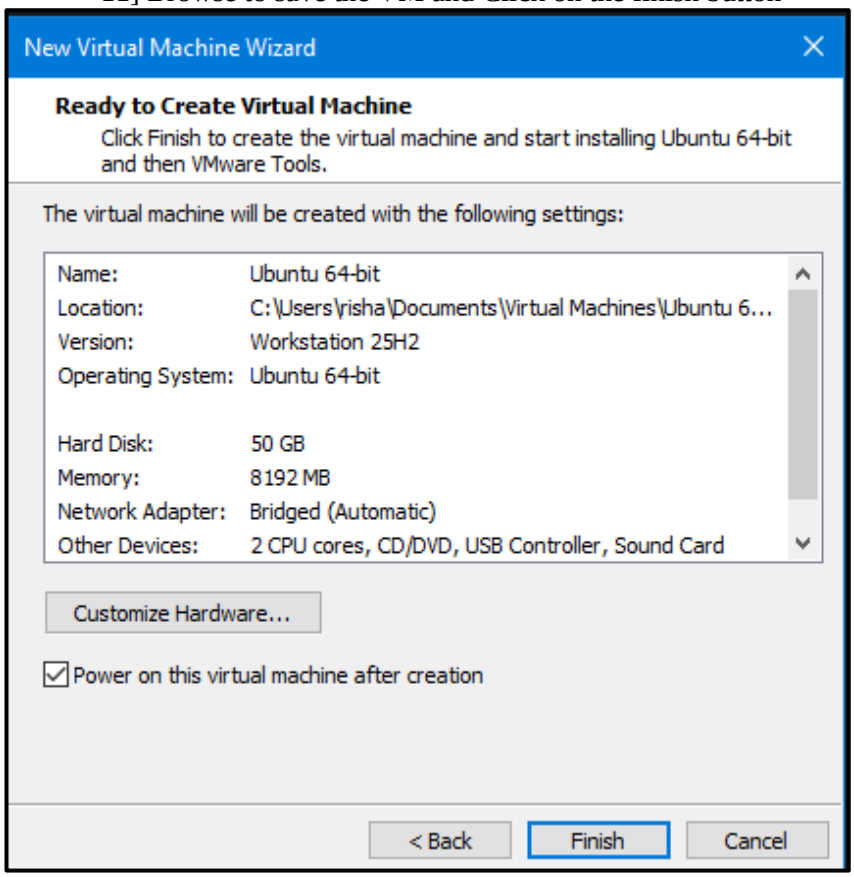


10] Change the capacity of the disk 50GB or Store virtual disk as a single file

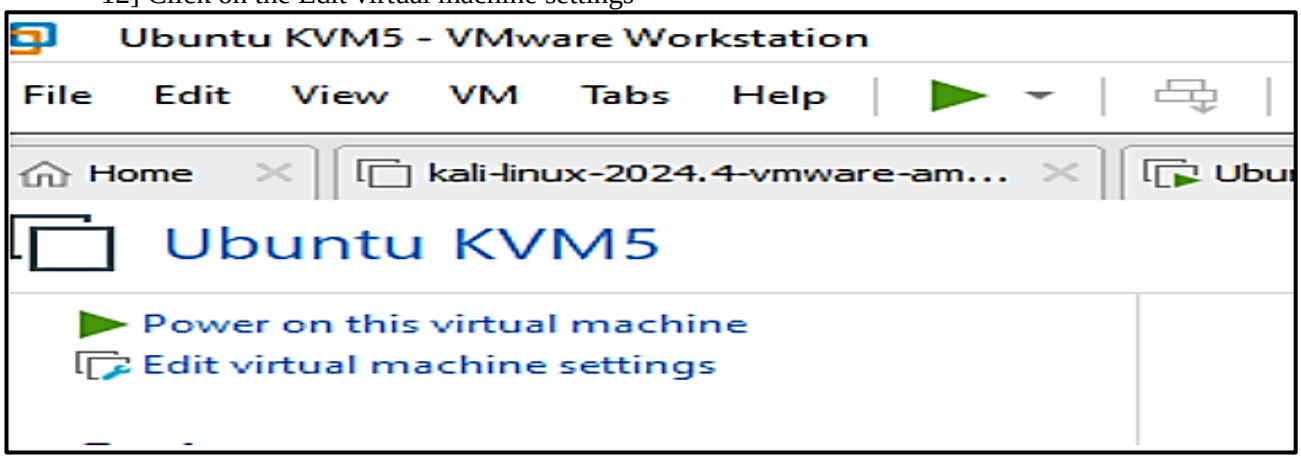




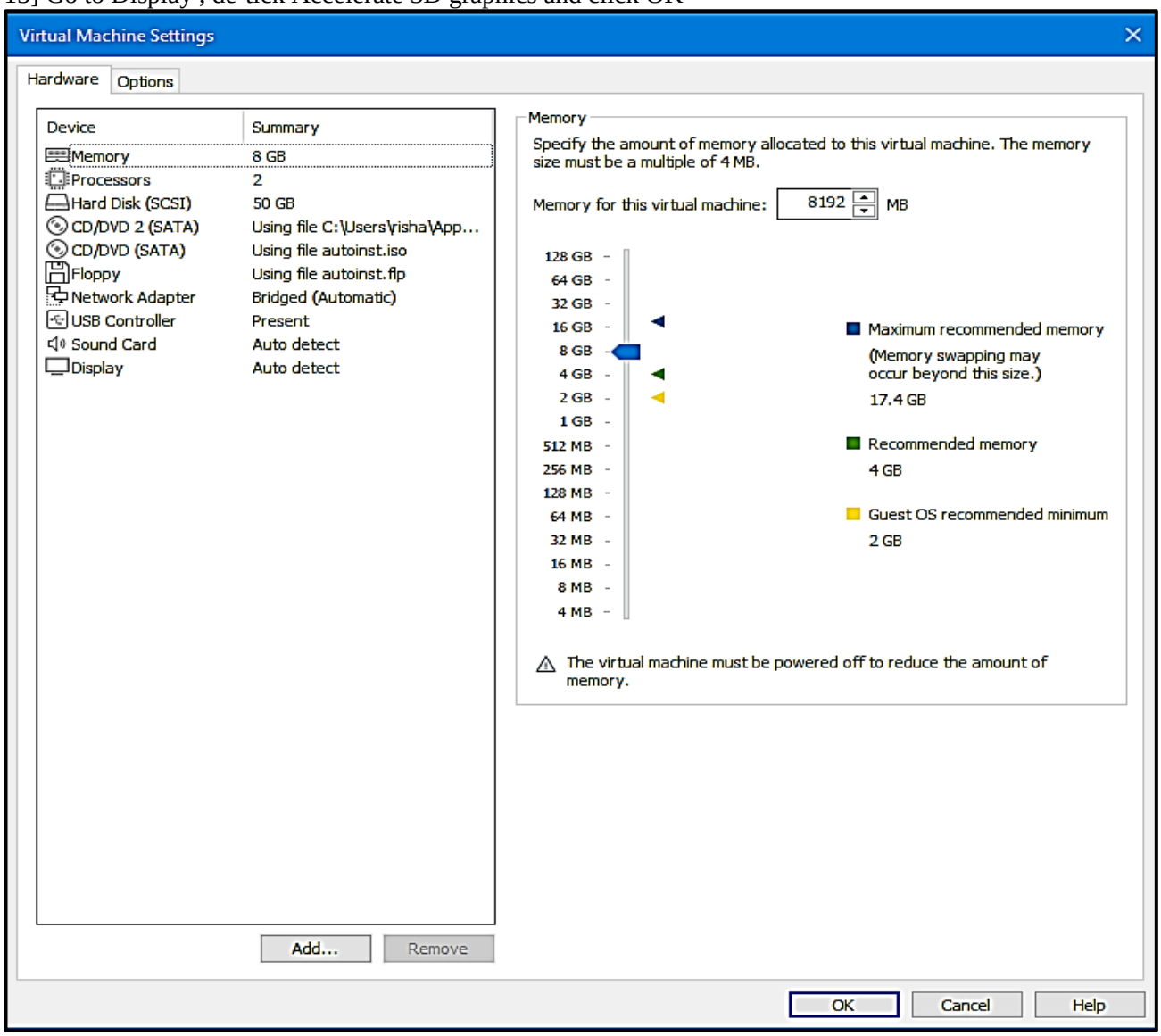
11] Browse to save the VM and Click on the finish button



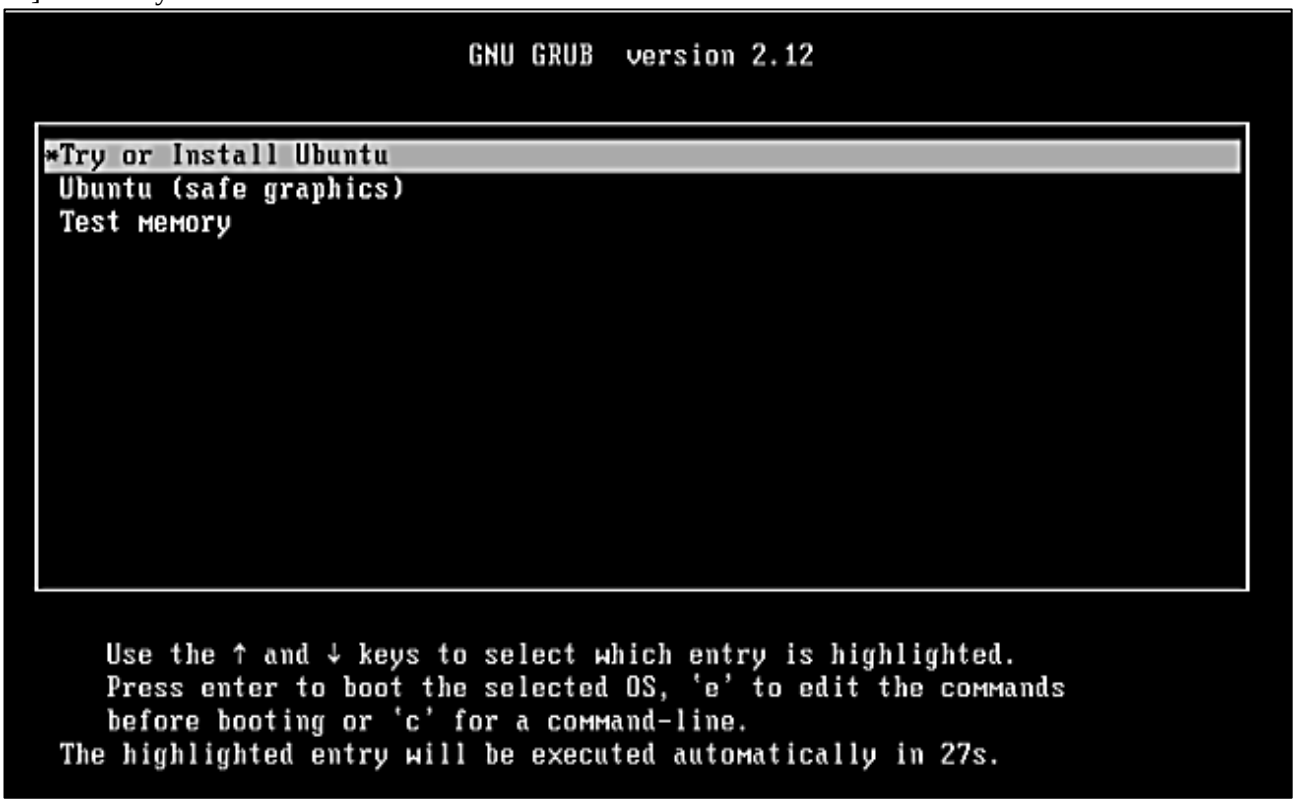
12] Click on the Edit virtual machine settings



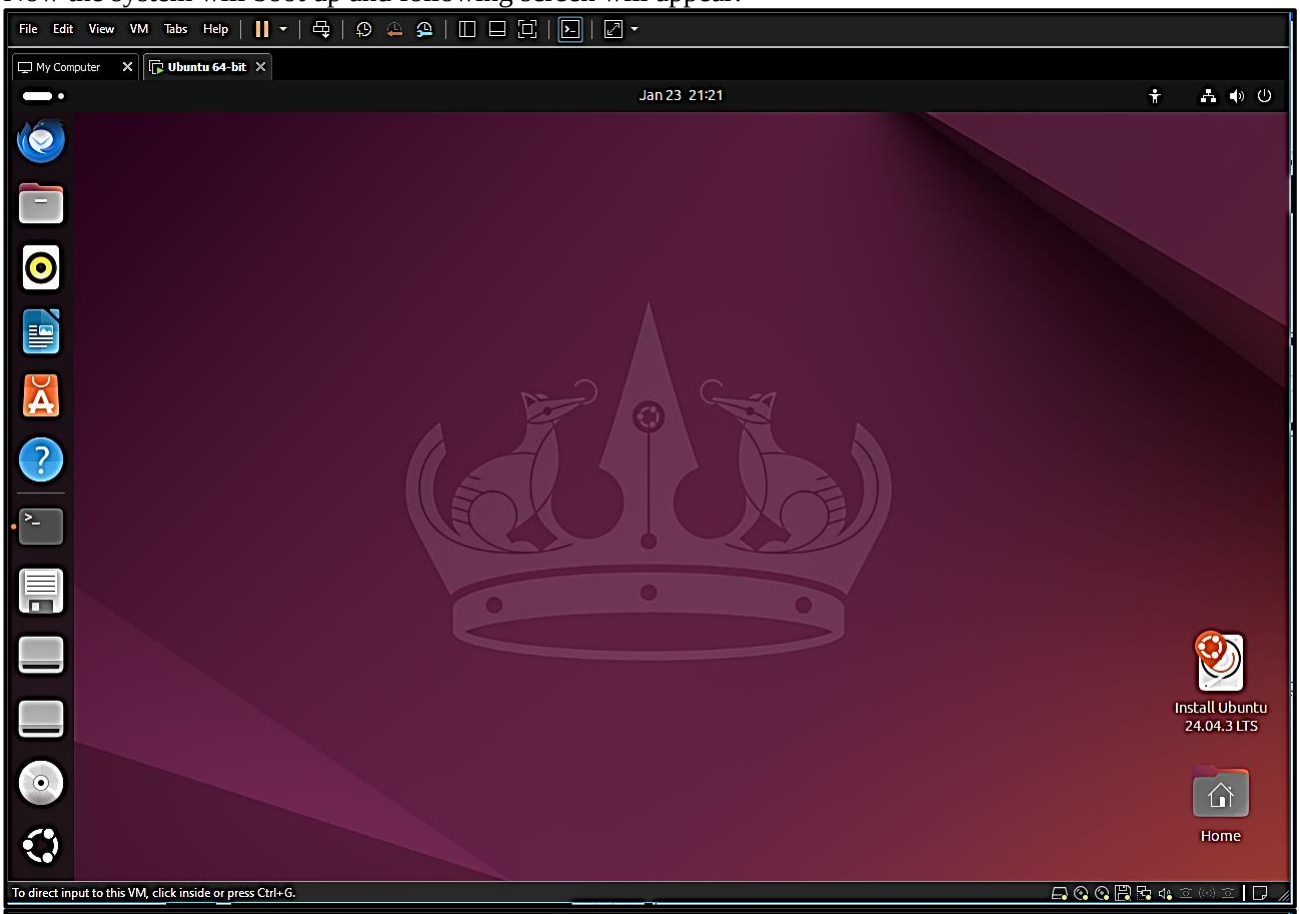
13] Go to Display , de-tick Accelerate 3D graphics and click OK

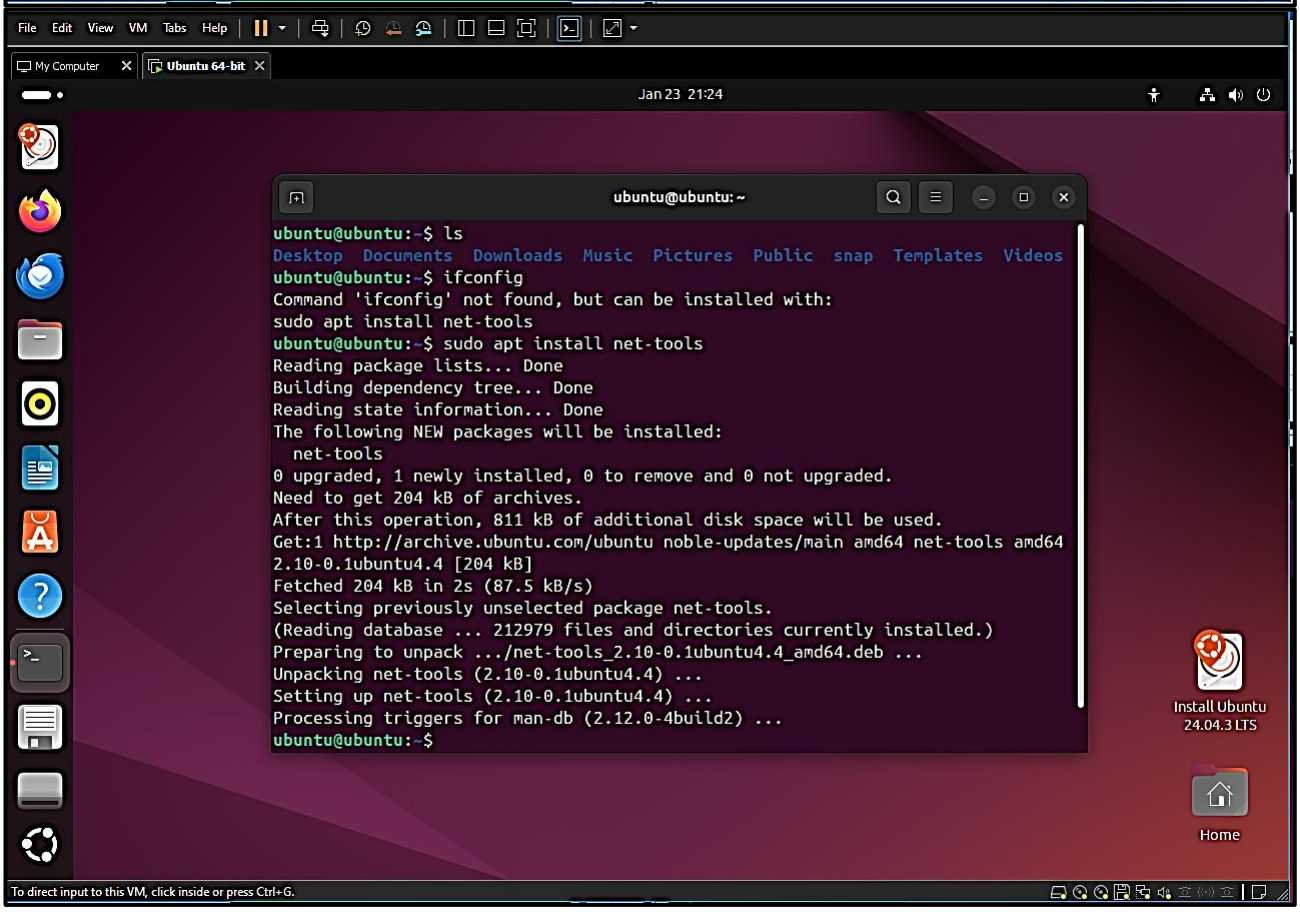


14] Select Try or Install Ubuntu



Now the system will boot up and following screen will appear:



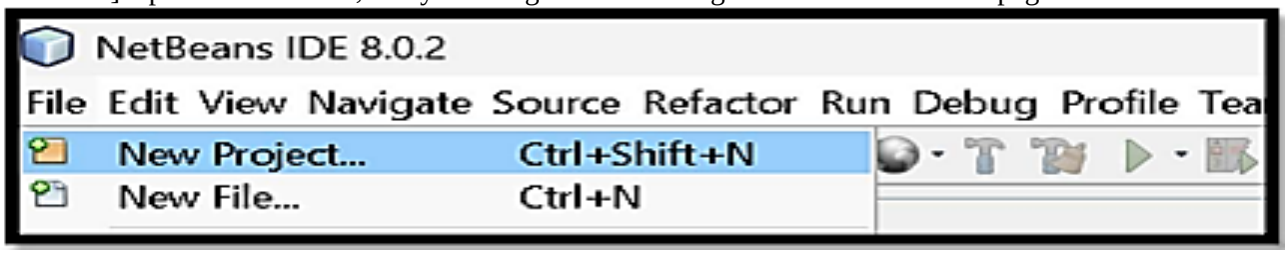


Practical 6

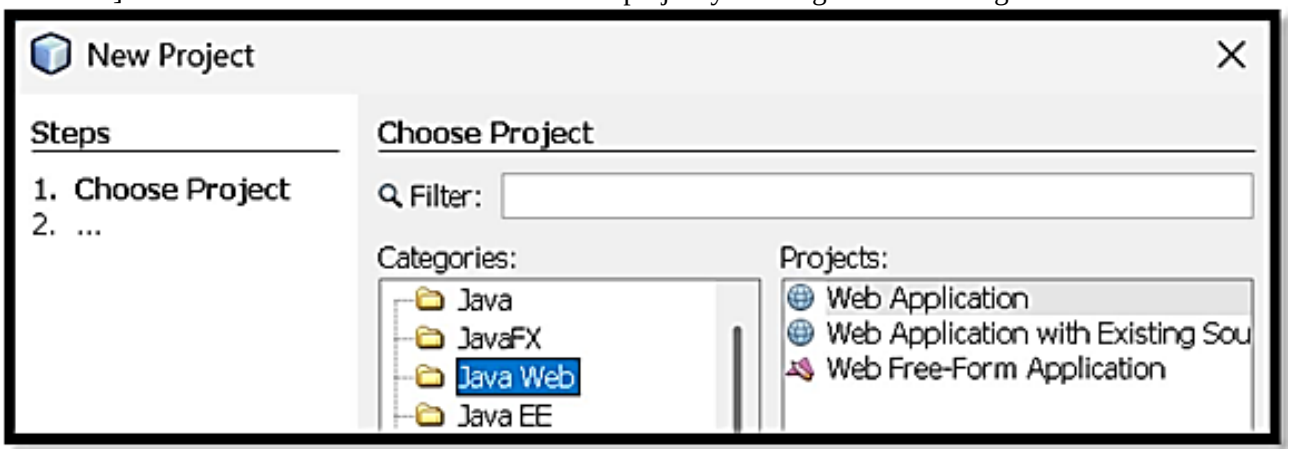
<u>Aim:</u> Develop application to download image/video from server or upload image/video to server using MTOM techniques.

<u>Steps:</u>

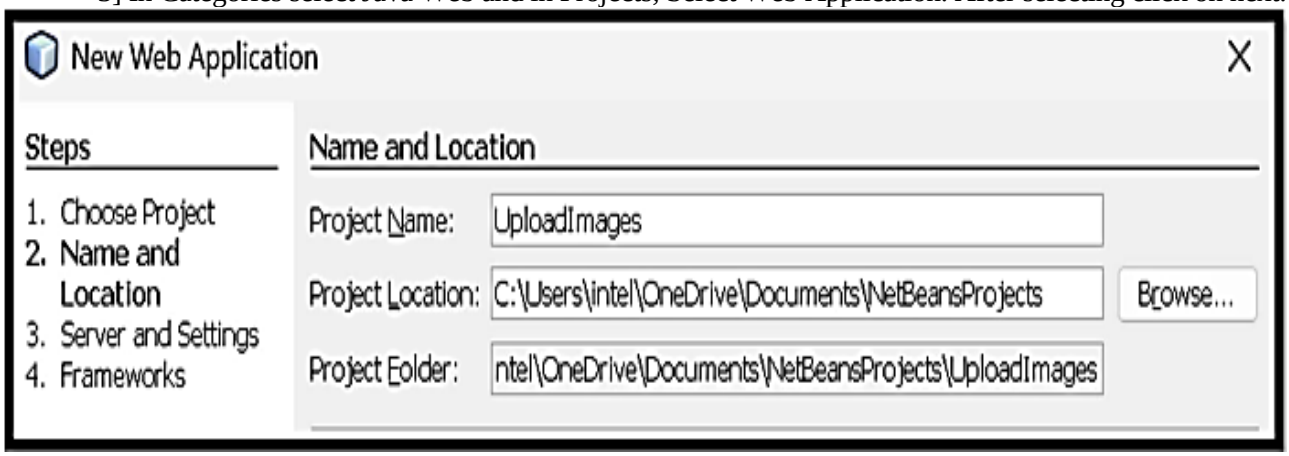
1] Open the NetBeans, and you will get the following screen. Close the start page.



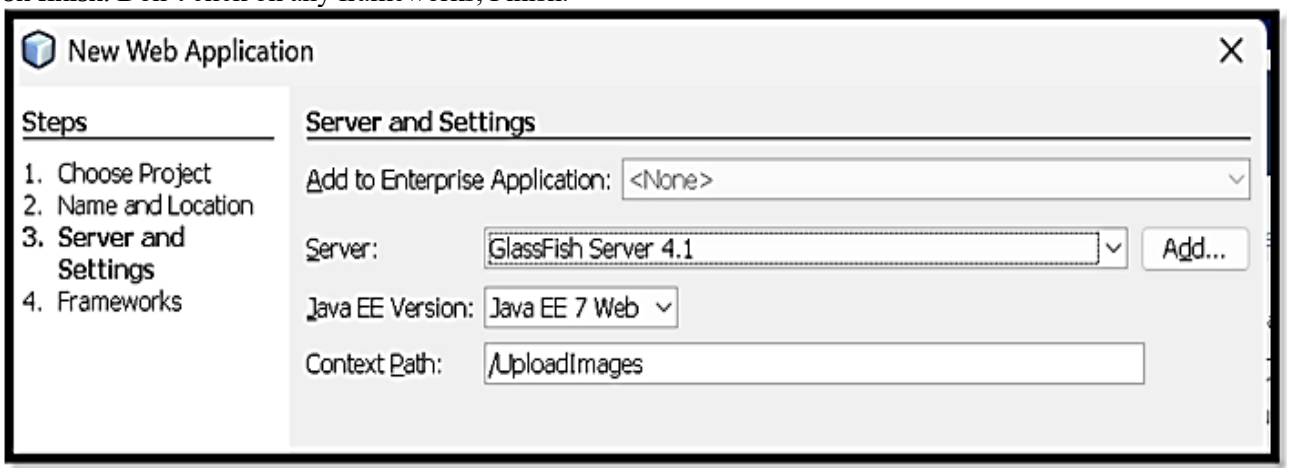
2] Now click on the file tab and click on new project you will get the following screen :



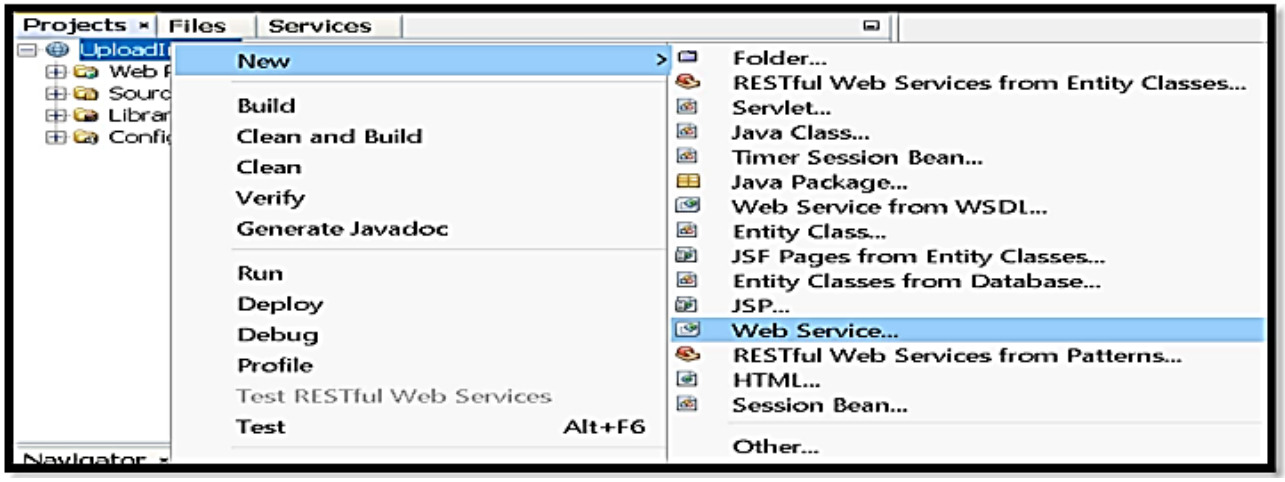
3] In Categories select Java Web and in Projects, Select Web Application. After selecting click on next.



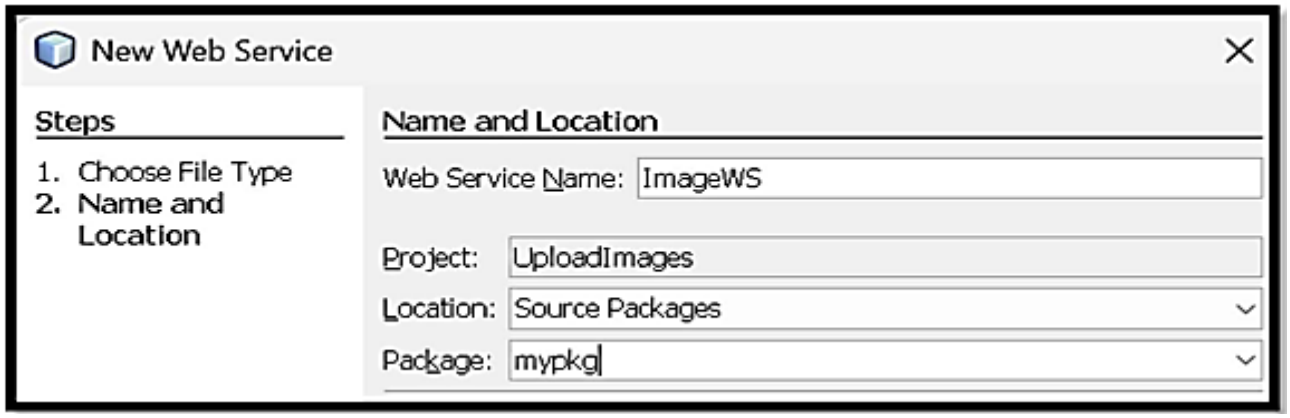
5] Now Give name to the Project Name: and click on next. you will get the following window then. again, click on finish. Don't click on any frameworks, Finish.



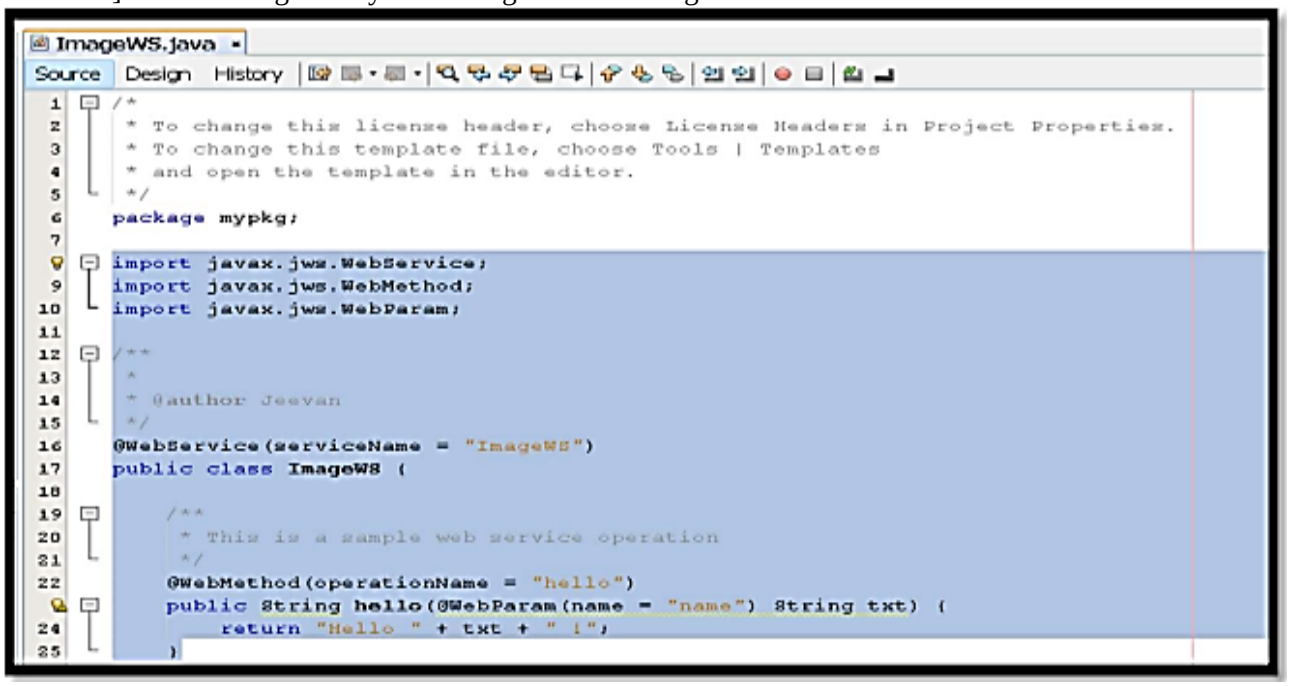
6] Now right click on the project and select new and then select Web Service, as shown Below:



7] After clicking Web Service following window should appear, now give name to web service and give package as “mypkg”. As shown in the image:



8] After clicking finish you should get the following window erase the mentioned code:



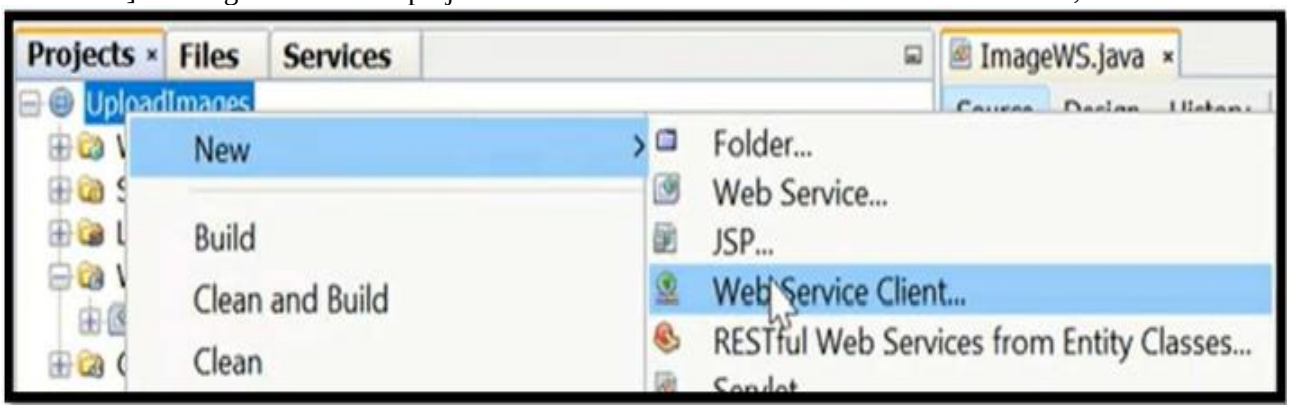
9] Now in ImageWS.java type this code:

```
import java.io.*;
import javax.jws.Oneway;
import javax.jws.WebService;
import javax.jws.WebMethod;
import javax.jws.WebParam;
import javax.xml.ws.soap.MTOM;

@MTOM(enabled = true, threshold = 6000)
@WebService(serviceName = "ImageWS")
```

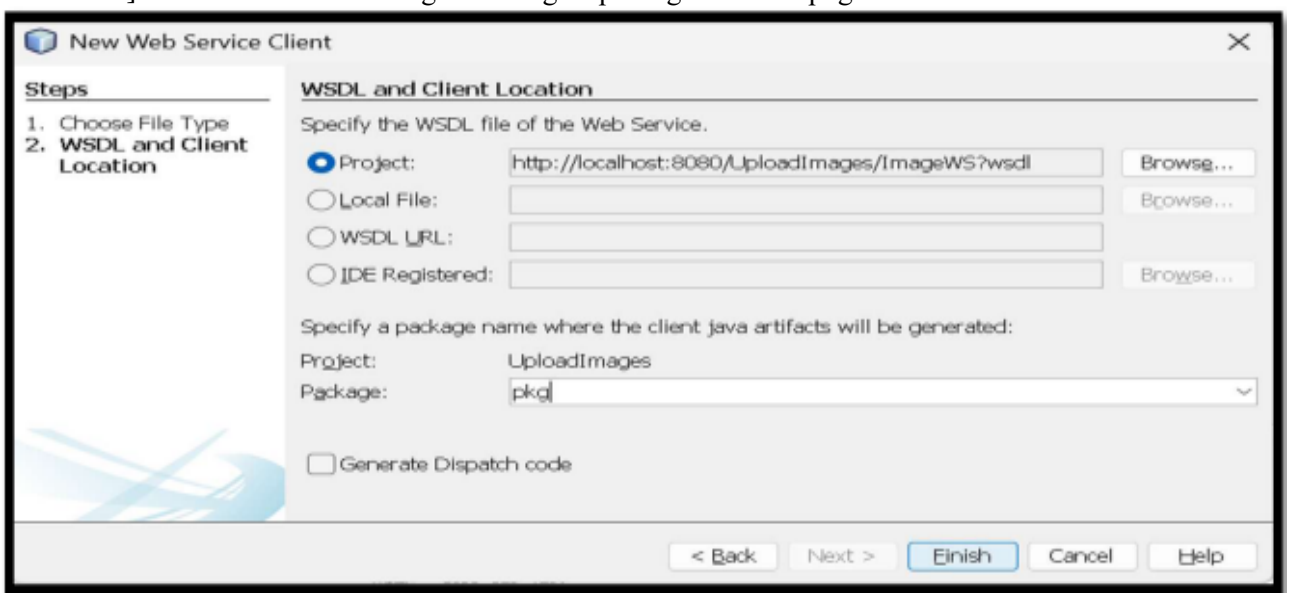
```
public class ImageWS {

    @WebMethod(operationName = "upload")
    @Oneway
    public void upload(@WebParam(name = "Filename") String Filename, @WebParam(name = "ImageBytes") byte[]
ImageBytes) {
        String filePath = "C:/Picture/upload/" + Filename;
        try {
FileOutputStreamfos = new FileOutputStream(filePath);
BufferedOutputStreambos = new BufferedOutputStream(fos);
bos.write(ImageBytes);
bos.close();
System.out.println("Received file: " + filePath);
        } catch (Exception ex) {
ex.printStackTrace();
        }
    }

    @WebMethod(operationName = "download")
    public byte[] download(@WebParam(name = "Filename") String Filename) {
        String filePath = "C:/Picture/upload/" + Filename;
System.out.println("Sending file: " + filePath);
        try {
            File file = new File(filePath);
FileInputStreamfis = new FileInputStream(file);
BufferedInputStream bis = new BufferedInputStream(fis);
byte[] fileBytes = new byte[(int) file.length()];
bis.read(fileBytes);
bis.close();
            return fileBytes;
        } catch (Exception ex) {
ex.printStackTrace();
            return null;
        }
    }
}
```

10] Now run file ImageWS.java.

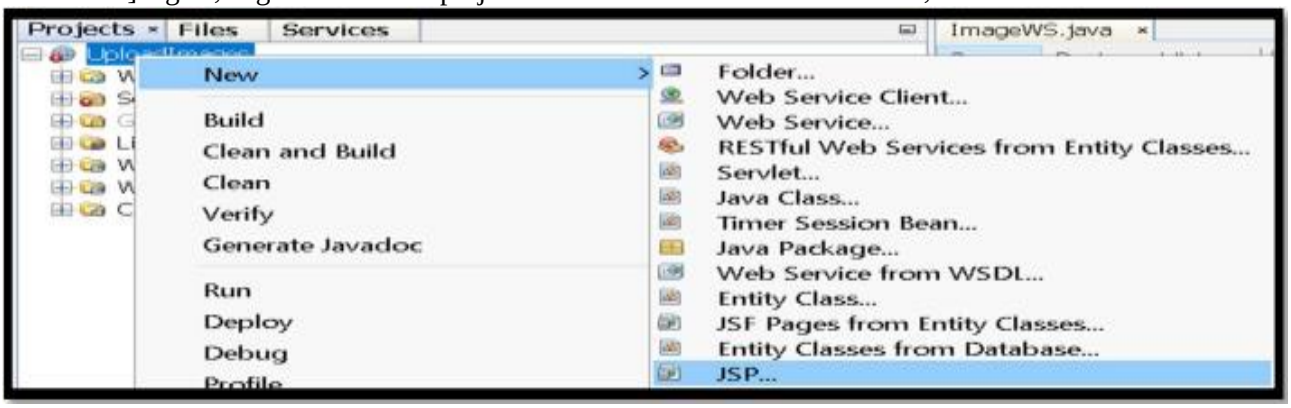
11] Now right click on the project and select new and then select Web Service Client, as shown Below:



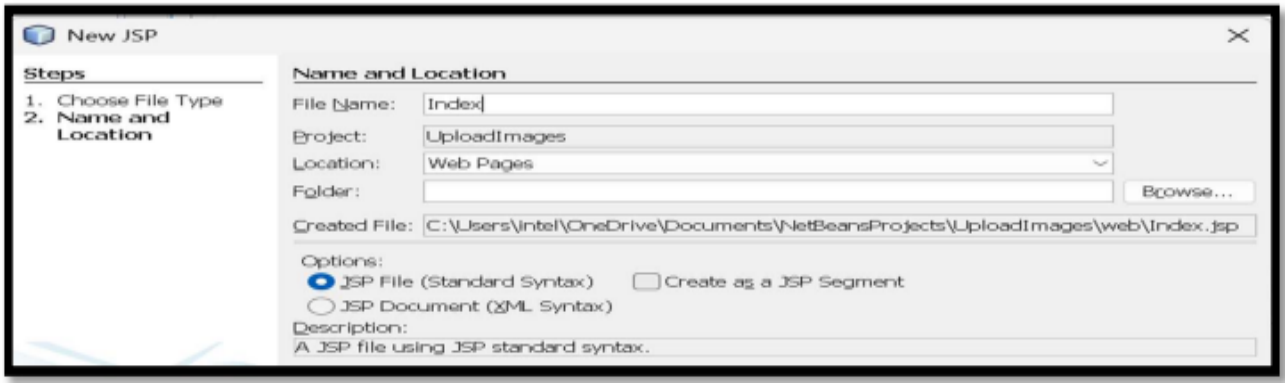
12] Then Browse select ImageWs and give package name is “pkg”.



13] Again, Right click on the project and select new and then select JSP, as shown Below:



14] Then file name is Index, as shown below:



15] Delete index.html file

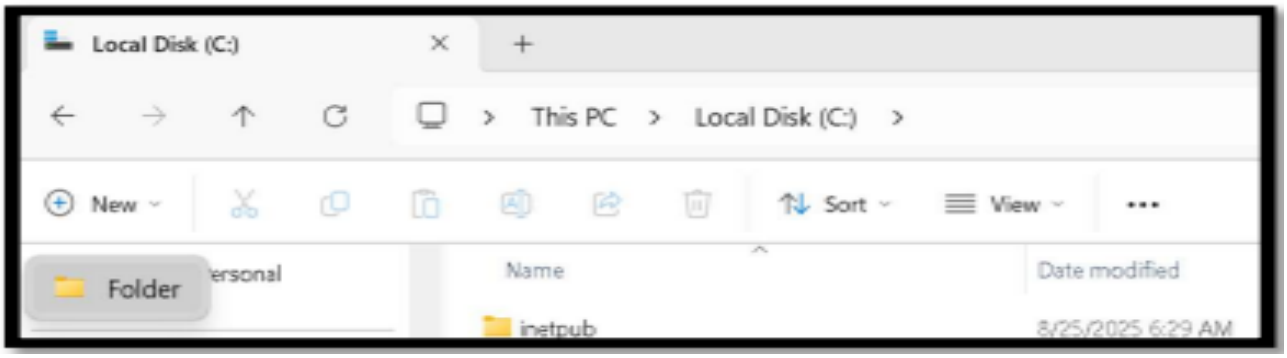
16] Now Erase all code index.jsp and type this code:

```
<%@page import="java.io.BufferedOutputStream"%>
<%@page import="java.io.FileOutputStream"%>
<%@page import="java.io.FileInputStream"%>
<%@page import="java.io.BufferedInputStream"%>
<%@page import="java.io.File"%>
<%@page import="javax.xml.ws.soap.MTOMFeature"%>
```

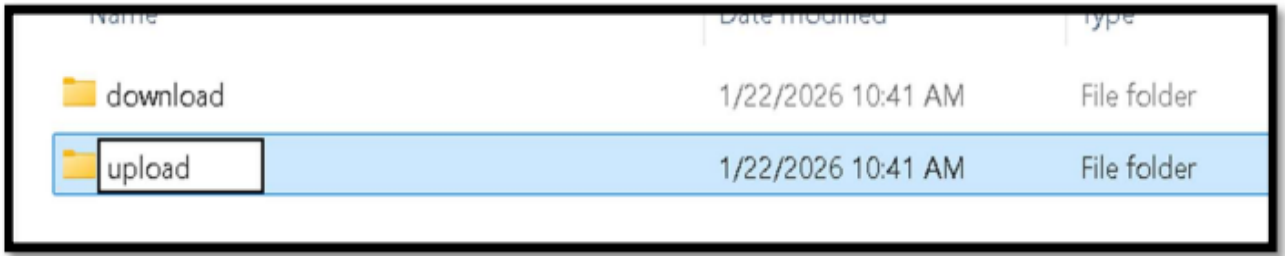
```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<title>JSP Page</title>
</head>
<body>
<%-- Start Web Service Innovation (Upload) --%>
<hr/>
<%
        try {
            // Initialize Service and Port with MTOM enabled
pkg.ImageWS_Service service = new pkg.ImageWS_Service();
pkg.ImageWS port = service.getImageWSPort(new MTOMFeature(6000));

            String filePath = "C:/Picture/car.jpg";
            File file = new File(filePath);

            // Read the file into a byte array
FileInputStreamfis = new FileInputStream(file);
BufferedInputStream bis = new BufferedInputStream(fis);
            String filename = file.getName();
byte[] imageBytes = new byte[(int) file.length()];

bis.read(imageBytes);
bis.close();

            // Call Web Service Upload
port.upload(filename, imageBytes);

out.println("File uploaded: " + filePath);

        } catch(Exception ex) {
ex.printStackTrace();
        }
      %>

<%-- End Web Service Innovation --%>
<hr/>
<%-- Start Web Service Innovation (Download) --%>
<hr/>

<%
        try {
            // Initialize Service and Port
pkg.ImageWS_Service service = new pkg.ImageWS_Service();
pkg.ImageWS port = service.getImageWSPort();

            String filename = "car.jpg";
            String filePath = "C:/Picture/download/" + filename;

            // Call Web Service Download
byte[] fileBytes = port.download(filename);

            // Write the downloaded bytes to a file
FileOutputStreamfos = new FileOutputStream(filePath);
BufferedOutputStreambos = new BufferedOutputStream(fos);

bos.write(fileBytes);
bos.close();
```

```
out.println("File download: " + filePath);

        } catch(Exception ex) {
ex.printStackTrace();
        }
    %>
</body>
</html>
```

17] Now make new folder in Local Disk, and rename it Picture, as shown below:

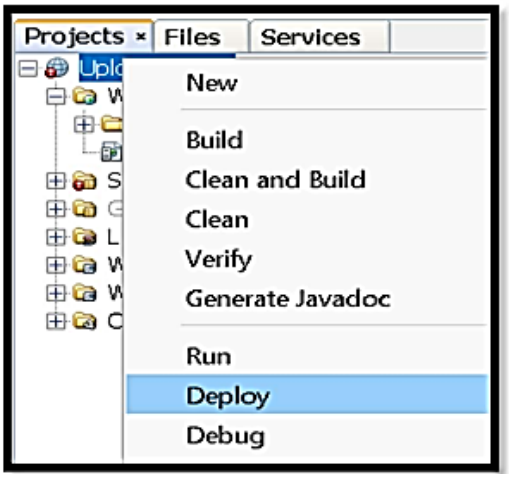


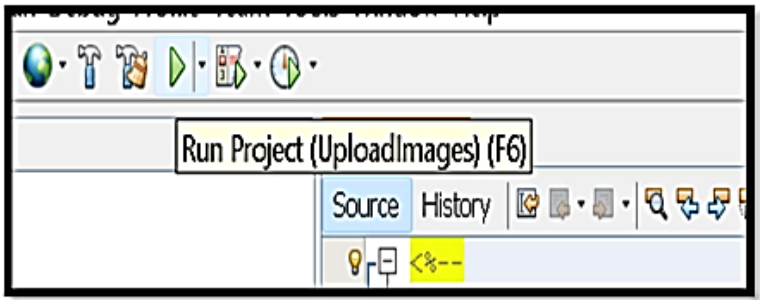
18] Now make new two folder inside in Picture folder name and rename it download and upload:



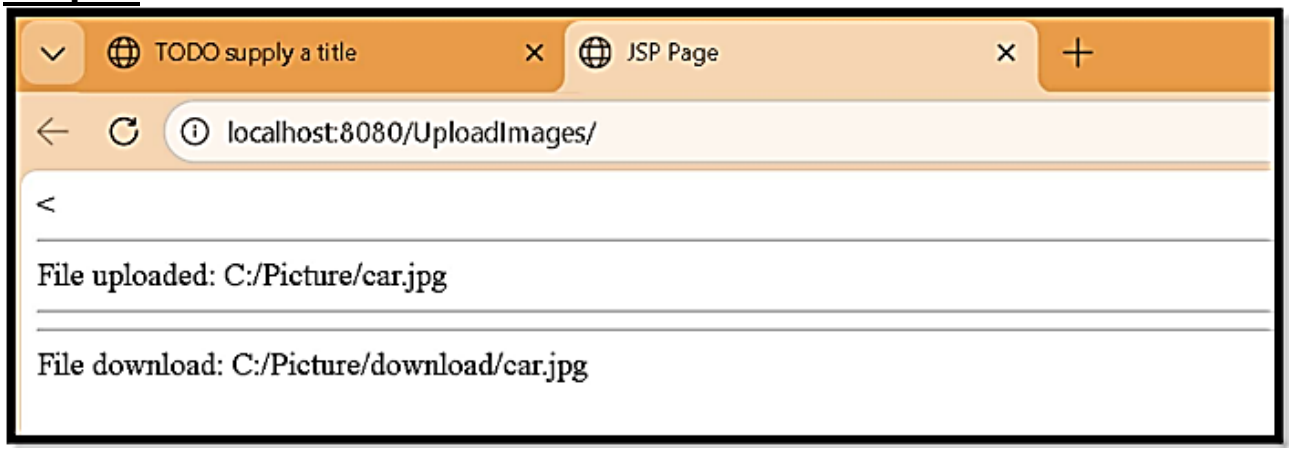
19] In picture folder paste one jpg image, as shown below:

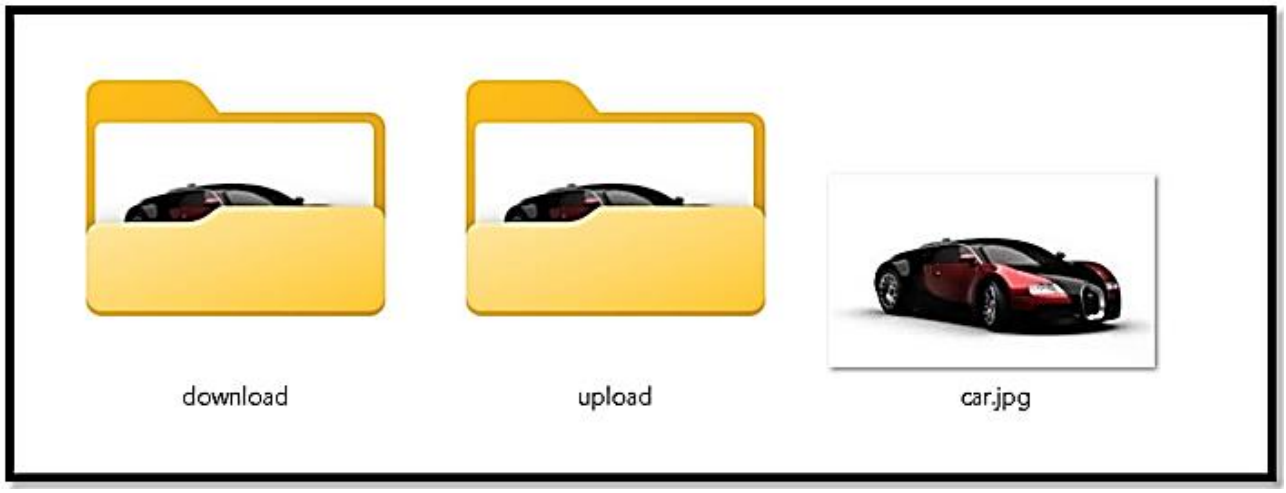
20 Now deploy project and run index.jsp file:





Output:


TODO supply a title
JSP Page
localhost:8080/UploadImages/
<
File uploaded: C:/Picture/car.jpg
File download: C:/Picture/download/car.jpg


download
upload
car.jpg

Practical 7

<u>Aim:</u> Using AWS Flow Framework develop application that includes a simple workflow. Workflow calls an activity to print hello world to the console. It must define the basic usage of AWS Flow Framework, including defining contracts, implementation of activities and workflow coordination logic and worker programs to host them.

AWS Flow Framework is a programming framework provided by Amazon Web Services that helps developers build distributed and asynchronous applications using workflows and activities. A workflow defines the coordination logic, while activities perform individual tasks. Workers are responsible for executing workflows and activities. In this practical, a simple workflow is implemented that invokes an activity to print a message on the console, demonstrating the basic usage of the AWS Flow Framework.

ALGORITHM

1. Define an activity interface.
2. Implement the activity to print “Hello World”.
3. Define a workflow interface.
4. Implement workflow logic to call the activity.
5. Create a worker program to execute the workflow.
6. Execute the program and verify output.

<u>Steps:</u>

1] Download VS Code and install JDK.
2] Create a Folder AWS with 5 java files (HelloActivity.java, HelloAtivityImpl.java, HelloWorkflow.java, HelloWorkflowImpl.java, Worker.java)

HelloActivity.java

```
public interface HelloActivity {
   void printHello();
}
```

HelloActivityImpl.java

```
public class HelloActivityImpl implements HelloActivity {
   public void printHello() {
System.out.println("Hello World from AWS Flow Activity");
   }
}
```

HelloWorkflow.java

```
public interface HelloWorkflow {
   void start();
}
```

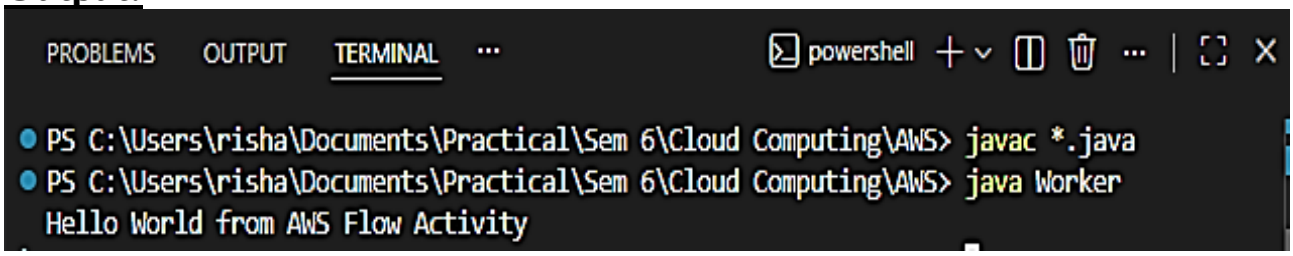
HelloWorkflowImpl.java

```
public class HelloWorkflowImpl implements HelloWorkflow {

HelloActivity activity = new HelloActivityImpl();

   public void start() {
activity.printHello();
   }
}
```

Worker.java

```
public class Worker {
   public static void main(String[] args) {
HelloWorkflow workflow = new HelloWorkflowImpl();
workflow.start();
   }
}
```

3] Open Terminal or press Ctrl+ ~ and then enter the following command to compile the java files:

```
javac *.java
java Worker
```

Output: